AI DRIVEN PRIVACY PRESERVING DIGITAL ECOSYSTEM FOR MODERNIZING SRI LANKA'S PADDY SUPPLY CHAIN

Panambara Arachchillage Neranda Dilhara Kumarasingha

(IT22294784)

Final (Draft) Report

Group ID: R26-SE-007

B.Sc. (Hons) Degree in Information Technology Specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology, Sri Lanka

October 2026

IOT BASED DIGITAL DASHBOARD FOR FADDY FARMING

Panambara Arachchillage Neranda Dilhara Kumarasingha

(IT22294784)

Final (Draft) Report

Group ID: R26-SE-007

Report submitted in partial fulfilment of the requirements for the B.Sc. (Hons) Degree in Information Technology Specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology, Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration.

I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure.

I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: P.A.N.D.Kumarasingha

Student ID: IT22294784

Signature:  Date: 10.10.2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: **Dr. Mahima Weerasinghe**		
Co-supervisor: **Mr. Eishan Weerasinghe**		

ABSTRACT

Modern agriculture depends a lot on farmers' gut feelings. Those gut feelings are not enough when weather shifts suddenly soil changes and new plant diseases appear. These problems cause farmers to miss the plant diseases, waste water and fertilizer and lose crop yields. Also the digital tools that exist today work in silos and they do not share data together.

To fix this gap this study suggests a Digital Farming system that mixes Deep Learning, real-time Internet of Things monitoring and cloud backups into a smooth flow. The Digital Farming system tackles three areas of rice farming: spotting disease, predicting yield and picking the best rice variety. For spotting disease a tuned ResNet50 neural network that runs on phone pictures reached 90.91% accuracy. It did better than models and only gave a warning when the confidence was 85% or higher so it avoids saying a plant is sick when it is not. For predicting yield, an XGBoost model that learned from land data and weather conditions scored 0.9890 on R^2 and had a mean error of 34.20 kg per hectare.

To keep the farm in view the Internet of Things data from ESP32 microcontrollers. A Firebase real-time database record temperature, humidity and soil moisture. The Internet of Things also has a weather API backup so the Deep Learning keeps running even if hardware breaks or the network stops. Also the Digital Farming system matches soil limits in each location to the rice types using prediction algorithms. The system runs on a FastAPI server keeps the XGBoost model in memory so it loads fast and answers quickly. In the end this research shows that mixing transfer learning, XGBoost and robust Internet of Things data gives farmers a strong data-based Precision Agriculture help system, for agriculture.

Keywords: Digital Farming, Internet of Things, Deep Learning, XGBoost, Precision Agriculture

ACKNOWLEDGEMENTS

I would like to take this opportunity to express my sincere gratitude to all those who contributed to the successful completion of my research on the Digital Farming Advisory Guidance System.

First and foremost, I extend my heartfelt appreciation to my supervisor, **Dr. Mahima Weerasinghe**, and my co-supervisor, **Mr. Eishan Weerasinghe**, for their continuous guidance, encouragement, and rigorous technical support throughout every stage of this project. Their profound expertise in Machine Learning, Computer Vision, and Internet of Things (IoT) architecture played a vital role in shaping my individual research. Their feedback directly assisted in optimizing the deep learning classification models and refining the hardware deployment strategies, ultimately molding this work into a well-structured and highly functional system.

I would also like to extend my sincere thanks to the Sri Lanka Department of Agriculture and local farming centers for providing crucial domain insights, historical yield datasets, and contextual field data. Their substantial support and contribution were invaluable in establishing a realistic testing environment, ensuring the algorithmic success and practical accuracy of this study.

Finally, I wish to acknowledge the Faculty of Computing and the Comprehensive Design and Analysis Project (CDAP) team at the Sri Lanka Institute of Information Technology (SLIIT) for providing me with the foundational resources, academic framework, and the opportunity to undertake this research.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.2 Individual component context 3
1.3 Problem statement 5
1.4 Research gap 6
1.5 Aim and objectives 9
1.6 Research questions or hypotheses 12
1.7 Scope limitations and assumptions 13
1.8 Individual contribution 14
1.9 Thesis structure 16

2 LITERATURE REVIEW AND RELATED WORK 17

2.1 Review method and scope 17
2.2 Domain and theoretical foundations 19
2.3 Related research 20
2.4 Existing systems products and techniques 21
2.5 Gap synthesis and conceptual framework 23

3 RESEARCH METHODOLOGY 25

3.1 Research design 25
3.2 Data participants or study materials 26
3.3 Experimental or study procedure 27
3.4 Measures and evaluation metrics 28
3.5 Data analysis 31
3.6 Ethics privacy security and responsible AI 32
3.7 Reproducibility and research artefacts 33

4 PRODUCT AND SOLUTION DESIGN 34

4.1 Requirements 34
4.2 Component architecture within the complete solution 35
4.3 Design alternatives and decisions 37
4.4 Detailed design 39
4.5 Standards and professional practice 42

5 IMPLEMENTATION INTEGRATION AND TESTING 43

5.1 Development environment and technology stack 43
5.2 Component implementation 45
5.3 Integration with the other components 48

5.4 Verification and software testing 50
5.5 Deployment and operation 52

6 RESULTS AND VALIDATION 54

6.1 Research results 54
6.2 Product and technical results 56
6.3 User or stakeholder validation 58
6.3.1 Farmer Validation Procedure 58
6.3.2 Farmer Feedback Results 60
6.3.3 Usability and Usefulness Evaluation 61
6.4 Integration results 64
6.5 Summary of findings 66

7 DISCUSSION 67

7.1 Interpretation of findings 67
7.2 Comparison with literature baselines and existing solutions 68
7.3 Research contribution 69
7.4 Product contribution 70
7.5 Limitations and threats to validity 71
7.6 Implications 72

8 CONCLUSION AND FUTURE DIRECTIONS 73

8.1 Conclusion 73
8.2 Achievement of objectives 74
8.3 Individual research and product contributions 75
8.4 Future directions 76

REFERENCES 77

GLOSSARY 79

APPENDICES 80

Appendix A Research instruments or extended results Supplementary research instruments and extended results referenced in the thesis include the farmer feedback questionnaire (Figure 11), farmer feedback summary (Table 8), and comparative model evaluation results (Tables 5 and 6). These materials are retained in their original locations in the main thesis. ..... **Error! Bookmark not defined.**

Appendix B Academic integrity and AI use disclosure The candidate declaration and academic integrity and AI disclosure are provided in the preliminary pages of this thesis. The original declaration is retained without modification. ..... **Error! Bookmark not defined.**

LIST OF FIGURES

Figure 1 - Example of an isolated commercial computer vision application for crop diagnostics. 21
Figure 2 - A traditional, passive IoT telemetry dashboard lacking integrated predictive machine learning 22
Figure 3 - Layered architecture of the Digital Farming Advisory Guidance System 35
Figure 4 - The comprehensive architectural flow bridging the FastAPI backend, ESP32 hardware, and ML pipelines 35
Figure 5 - Environment, action, state and data-flow definition of the advisory system 37
Figure 6 - ResNet50 image classification and confidence-based decision workflow 39
Figure 7 - Photograph of the assembled IoT research prototype used during component development 45
Figure 8 - Normal ESP32 telemetry-to-yield-prediction processing workflow 45
Figure 9 - Hardware-disconnection and external weather API fallback workflow ... 50
Figure 10 - Mean Farmer Feedback Ratings Across Seven System Evaluation Criteria 62
Figure 11 - Farmer Feedback Questionnaire for User and Stakeholder Validation of the Paddy Farming Advisory Guidance System 63

LIST OF TABLES

Table 1 - List of Abbreviation..... viii
Table 2 - Research Gap of the Proposed System 8
Table 3 - Principal technologies and their responsibilities in the individual research component. 43
Table 4 - Verification evidence and remaining limitations for the implemented component 50
Table 5 - Comparative evaluation of deep learning architectures for paddy leaf disease classification. 54
Table 6 - Comparative evaluation of machine learning regressors for paddy yield prediction..... 55
Table 7 - Proposed Farmer Validation Study Design 60
Table 8 - Illustrative Farmer Feedback Summary (n = 10)..... 60

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
ResNet50	Residual Network with 50 Layers
XGBoost	Extreme Gradient Boosting
RF	Random Forest
DT	Decision Tree
GAP	Global Average Pooling
ReLU	Rectified Linear Unit
IoT	Internet of Things
ESP32	Espressif Systems 32-bit Microcontroller
DHT22	Digital Temperature and Humidity Sensor
ADC	Analog-to-Digital Converter
GPIO	General-Purpose Input/Output
GPS	Global Positioning System
Wi-Fi	Wireless Fidelity
API	Application Programming Interface
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
MSE	Mean Squared Error
RMSE	Root Mean Squared Error
MAPE	Mean Absolute Percentage Error
R^2	Coefficient of Determination

Table 1 - List of Abbreviation

1 INTRODUCTION

1.1 Overall project context

Sri Lanka's paddy sector plays a significant role in the country's agricultural economy and food supply system. However, the modernization of the paddy supply chain requires more intelligent, secure, and privacy-preserving digital technologies capable of supporting farmers and other stakeholders across different stages of the agricultural and market ecosystem. Existing technological approaches generally address individual aspects of the paddy supply chain, such as crop monitoring, yield prediction, price forecasting, traceability, or online marketplaces, without providing an integrated ecosystem that combines intelligent decision-making, privacy protection, secure transactions, and stakeholder interaction.

The proposed research, "**AI Driven Privacy Preserving Digital Ecosystem for Modernizing Sri Lanka's Paddy Supply Chain**," addresses this gap through an integrated ecosystem consisting of four complementary components. The first component focuses on developing a Digital Dashboard for paddy farming, combining real-time data, deep learning, Vision Transformers, Retrieval-Augmented Generation (RAG), and Graph Neural Networks (GNNs) to support yield prediction, crop image analysis, and knowledge-augmented agricultural advisory services. The research specifically identifies the lack of systems that integrate multimodal agricultural inputs such as images, soil, weather, and farm metadata, as well as the limited application of Digital Dashboard, RAG, and GNNs within the Sri Lankan paddy farming context.

The second component addresses the uncertainty associated with paddy price forecasting. Conventional statistical and machine-learning approaches can predict prices, but they provide limited understanding of the underlying causes of price changes. Therefore, the proposed component combines Transformer-based time-series forecasting with Causal Analysis Learning, Large Language Model reasoning.

The third component focuses on establishing privacy-preserving and trustworthy supply-chain traceability. Although blockchain can provide tamper-resistant records for tracking agricultural products, conventional blockchain-based approaches may expose sensitive information about farmers and other participants. To address this

limitation, the proposed ecosystem incorporates Zero-Knowledge Proofs (ZKPs), Decentralized Identity (DID), smart contracts, and AI-based fraud detection using Graph Machine Learning. These technologies are intended to enable stakeholders to verify relevant agricultural and transaction information without unnecessarily exposing sensitive data, while also supporting secure identities and the detection of fraudulent or collusive behaviour within the supply chain.

The fourth component introduces an AI-powered farmer–miller marketplace designed to improve market participation and decision-making while preserving stakeholder privacy. Unlike conventional agricultural marketplaces that primarily display prices, the proposed approach incorporates RAG-based LLM assistants, Federated Learning, secure matching, and multi-agent negotiation. Federated Learning enables AI models to benefit from distributed farmer data without requiring the underlying private data to be centrally shared. Secure matching mechanisms can further protect sensitive information during buyer–seller interactions, while AI agents can support automated negotiation between farmers and millers.

Collectively, these four components establish a unified AI-driven, privacy-preserving digital ecosystem for Sri Lanka's paddy supply chain. The ecosystem aims to connect farm-level intelligence, market forecasting, secure and privacy-preserving traceability, and intelligent farmer–miller interactions within a common technological framework. By integrating Artificial Intelligence, Digital Dashboard, XGBoost-based forecasting, Causal Anaysis Learning, RAG, Graph Neural Networks, Blockchain, Zero-Knowledge Proofs, Decentralized Identity, Federated Learning, secure matching, and multi-agent systems, the research seeks to move beyond isolated agricultural applications toward a more comprehensive digital ecosystem capable of supporting informed decision-making, trustworthy transactions, privacy protection, and efficient stakeholder interaction across the paddy supply chain.

1.2 Individual component context

Within the broader scope of the integrated smart agricultural platform, this specific individual component the Farmer Advisory Guidance System focuses exclusively on the computational intersection of precision agriculture, deep learning, and Internet of Things (IoT) telemetry. Traditional rice cultivation is historically dependent on heuristic knowledge and manual observation. This component fundamentally shifts that paradigm by engineering a highly automated, data-driven decision support system capable of analyzing both macroscopic geographical constraints and microscopic field anomalies in real time.

The contextual foundation of this component rests upon three distinct predictive domains: optimal varietal selection, pathological disease diagnosis, and environmental yield forecasting. To mitigate the risk of planting genetically unsuitable crops, the system employs Scikit-Learn classification pipelines—specifically utilizing Random Forest ensembles and mathematical One-Hot Encoding algorithms. This allows the component to map complex geographical attributes, such as soil salinity levels and iron toxicity risks, to specific rice variety codes, immediately providing farmers with enriched botanical data such as expected growth durations and flood tolerances. Furthermore, to achieve pathological identification, the component leverages advanced computer vision via a fine-tuned ResNet50 Convolutional Neural Network (CNN). By utilizing transfer learning with frozen ImageNet base layers, this CNN processes mobile leaf image uploads to accurately classify devastating fungal and bacterial infections such as Bacterial leaf blight and Brown spot while strictly enforcing an 85% mathematical confidence gating mechanism to filter out irrelevant anomalous images.

Simultaneously, the component addresses the volatile nature of crop yield via an eXtreme Gradient Boosting (XGBoost) regression pipeline. Because traditional yield predictions rely heavily on static historical data, they often fail to account for sudden climatic shifts. To resolve this contextually, the regression model is deeply integrated with a physical edge-computing layer. ESP32 microcontrollers deployed directly in the paddy fields continuously push live telemetry including ambient temperature, relative humidity, and capacitive soil moisture to a Firebase Realtime Database. The

FastAPI backend actively queries this IoT stream, allowing the XGBoost regressor to continuously adjust its yield estimations based on the exact, current physical state of the field. [6] [12] [13]

Furthermore, the operational context of this component mandates extreme computational resilience. Because agricultural IoT hardware is perpetually exposed to severe weather, the component incorporates a dynamic, fault-tolerant fallback protocol. Should the physical ESP32 nodes disconnect or fail, the asynchronous backend autonomously detects the null payload and immediately pings external geolocation weather APIs to retrieve substitute macro-environmental data, ensuring that the machine learning pipelines experience zero downtime. Ultimately, the context of this individual component is to act as the primary analytical engine of the overarching project, translating chaotic environmental variables and raw image tensors into scientifically validated agronomic intelligence.

1.3 Problem statement

In the modern agricultural sector, traditional farming practices continue to rely predominantly on manual intuition and localized heuristic knowledge. However, this non-scientific approach is becoming increasingly inadequate in the face of rapid climate volatility, sudden soil degradation, and the emergence of devastating crop diseases. When farmers select rice varieties based purely on tradition rather than mathematically mapping soil toxicity and geographic salinity, they inadvertently expose their crops to massive vulnerabilities. Furthermore, when fungal and bacterial leaf diseases such as Bacterial leaf blight or Brown spot are manually misdiagnosed in the field, inappropriate chemical treatments are often applied, resulting in severe ecological damage and catastrophic crop failure.

While the introduction of digital agriculture has attempted to address these issues, existing solutions present severe operational limitations. Current tools typically operate in isolated computational silos; farmers are forced to use one application for weather forecasting, a separate platform for pest identification, and disjointed historical data for yield estimation. There is a critical lack of a unified analytical pipeline capable of processing multi-modal agricultural data simultaneously. More importantly, systems that do utilize real-time Internet of Things (IoT) sensors are fundamentally fragile. Agricultural environments are aggressive, often causing field sensors to lose power or cellular connectivity. If a predictive machine learning model relies solely on this hardware telemetry, a physical sensor failure instantly triggers runtime exceptions, rendering the entire digital advisory system completely useless.

Therefore, the primary research problem addressed by this component is: How can deep learning computer vision algorithms (for rapid disease diagnosis) and second-order gradient boosting regressors (for yield forecasting) be optimally integrated with real-time IoT hardware into a single, cohesive cloud architecture? Specifically, how can this system be mathematically engineered to maintain continuous computational availability and uninterrupted model inference via dynamic cloud API fallbacks even in the event of complete physical hardware failure in the field?

1.4 Research gap

Although existing smart agriculture and digital farming studies have introduced important advances in crop monitoring, disease classification, and yield estimation, the literature still shows several gaps when these solutions are examined from the perspective of a complete end-to-end digital advisory workflow for rice farmers. The following discussion summarizes the key limitations identified in five related agricultural studies and shows how these gaps justify the need for the proposed system.

Research 01

The paper focuses on deep learning architectures using Convolutional Neural Networks (CNNs) for diagnosing plant leaf diseases from digital photographs. Its main strength lies in achieving strong image classification accuracy for visual symptoms under controlled laboratory settings. However, the work does not incorporate multimodal inputs by pairing visual classification with real-time environmental metrics like ambient temperature, relative humidity, and soil moisture. It also lacks explicit probability confidence thresholding (gating), which causes the model to output false positive diagnoses when non-crop images or ambiguous photos are uploaded. Furthermore, the study omits real-time yield forecasting (kg/ha) and does not connect visual disease identification to IoT hardware streams or fault-tolerant fallback mechanisms. Therefore, while the study contributes to image recognition, it does not address the broader operational decision-support pipeline required by farmers.

Research 02

The research proposes sensor-based environmental monitoring setups for agricultural fields using microcontroller units like ESP32 and Arduino paired with wireless transmission modules. Although this contributes to field-level environmental tracking, the work does not include automated computer vision for crop disease identification or varietal selection models tailored to local soil parameters. It also misses integrated predictive models to estimate pre-harvest yields based on incoming environmental sensor data. Furthermore, the study does not incorporate dynamic fail-safe protocols such as automated third-party weather API fallbacks to handle hardware failures,

sensor dropouts, or network loss in remote fields. As a result, the system remains focused purely on passive telemetry collection rather than intelligent, uninterrupted decision support.

Research 03

The study presents yield estimation frameworks using traditional machine learning regressors and statistical historical data, primarily aimed at macro-level agricultural forecasting. This is valuable for regional agricultural planning and policy development. However, the study neglects real-time field-level environmental telemetry, relying exclusively on static historical averages that fail to capture micro-climatic shifts. It also lacks a real-time computer vision component for identifying fungal or bacterial leaf infections in the field. In addition, the paper does not incorporate automated rule-based agronomic decision engines to translate numeric regression predictions into actionable field instructions (such as immediate irrigation alerts). Hence, its contribution is primarily retrospective and macro-level, lacking proactive, field-level farmer guidance.

Research 04

The research applies deep learning and computer vision to identify crop stress and nutrient deficiencies from drone and satellite imagery. This makes it useful for macroscopic farm observation. However, the approach operates separately from ground-level IoT sensing networks and localized mobile interaction, making it inaccessible for everyday smallholder farmers requiring instant on-the-ground feedback. The paper also does not extend its vision outputs into non-linear yield regression models (kg/ha), nor does it integrate geographical soil selection rules for rice varieties. Consequently, although the study is effective in remote sensing, it does not provide the integrated digital farming workflow needed to connect ground-level disease diagnosis, micro-climatic tracking, and localized yield management.

Research 05

The paper is a review that surveys general IoT-enabled decision support systems in precision agriculture. While the survey is useful for understanding overarching digital farming frameworks, it does not offer a technical, deployed implementation connecting computer vision, gradient-boosted regression, and hardware sensors within a unified asynchronous service architecture. It also overlooks the use of asynchronous in-memory model loading to resolve thread-locking latency during high-frequency API requests, and omits dynamic cloud fallbacks for handling hardware outages in real time. Therefore, the survey does not capture the practical, fault-tolerant engineering required for real-world agricultural decision support.

Feature / Capability	**Research 01**	**Research 02**	**Research 03**	**Research 04**	**Research 05**	**Proposed System**
Multimodal Inputs (Vision + Environmental)	No	No	No	No	No	**Yes**
CNN Transfer Learning with Confidence Gating ($\geq 85\%$**)**	Yes	No	No	Yes	No	**Yes**
Non-Linear Yield Regressor (XGBoost, kg/ha**)**	No	No	Yes	No	No	**Yes**
Real-Time ESP32 Field Telemetry Integration	No	Yes	No	No	No	**Yes**
Automated Dynamic Cloud Weather API Fallback	No	No	No	No	No	**Yes**
Soil-to-Varietal Predictive Mapping	No	No	No	No	No	**Yes**
Rule-Based Agronomic Action Engine	No	No	No	No	No	**Yes**
In-Memory Asynchronous API Optimization ($\mathcal{O}(\text{compute})$**)**	No	No	No	No	No	**Yes**

Table 2 - Research Gap of the Proposed System

1.5 Aim and objectives

Primary Aim

The main goal of this research part is to create, build and mathematically prove a strong and integrated system for guiding farmers. This system wants to replace the way of farming that depends on guesswork. Instead it gives farmers real-time information that is specific to their exact location. It connects deep learning models with continuous edge-computing from IoT devices and a flexible cloud setup. The final result should be one platform that helps farmers make better decisions. This platform must be able to do things at once: detect difficult plant diseases predict how much crops will yield using changing weather and soil conditions and suggest the best rice types based on the local geography. Importantly the system must stay fully working all the time. Even if the hardware, in the field breaks or fails the machine learning models should keep running without interruption.

Specific Objectives

To successfully achieve the primary aim, the research is divided into four highly specific, measurable, and technically rigorous objectives :

1. To build a Deep Learning Convolutional Neural Network (CNN) that can diagnose diseases in time. The first goal is to reduce crop loss by identifying diseases with high accuracy. The aim is to test deep learning models including MobileNetV2, EfficientNetB0 and ResNet50 to work with image data that comes from smartphones. This goal needs the use of Transfer Learning methods by locking the basic ImageNet weights of the ResNet50 model so that it can find small important features of fungal and bacterial diseases, like Bacterial leaf blight and Brown spot without needing too much computer power. A key part of this is putting in a math rule using the Softmax activation. By setting a minimum of 85% confidence level the system must automatically remove strange images so that wrong results are not given and the diagnosis is always correct.

2. To create a Yield Prediction model that's dynamic and responsive. Traditional ways of predicting yield use averages that don’t consider sudden changes in the weather. So the second goal is to build a -linear regression system that can predict how much rice will be produced in kilograms per hectare. This system will use land data and current changing weather conditions. The process includes testing decision trees and Random Forests against XGBoost algorithms. The main goal is to use XGBoost’s second-order math and its built-in L1 and L2 regularization to stop the model from learning much from the messy agricultural data. In the end the system needs to make predictions that change based on geographic details like District and Paddy Type. These details are made consistent using a StandardScaler. The system also connects with real-time information, like temperature, humidity and soil moisture at the time. [6] [7]

3. To mathematically connect geography and soil conditions to the rice varieties. Planting crops that're not suitable for the genetics of the area greatly lowers how much can be produced. So the third goal is to build a classification module using Scikit-Learn that connects complicated environmental factors like when the season starts the type of area exact places, salt levels and risks from too much iron, to specific rice

variety codes. Since these inputs are categories this goal needs a way to prepare the data using a ColumnTransformer and OneHotEncoder to change words into numbers. The goal also needs a way to look up information in a dictionary that is stored in memory. This dictionary checks the number code against a list of rice varieties that is saved as a file called RiceVarietal_Category.csv. This way the person using the system gets useful information like how long the plant takes to grow and how well it can handle water instead of just a number. [11]

4. To design a system that can keep working when things go wrong especially for the Internet of Things Cloud Backend. The important and complicated part is putting the system into real life and making all the pieces fit together. This means building computer units called edge nodes using ESP32 chips. These chips are placed in covers that protect them from rain and sun. They have sensors that measure temperature and humidity and also check how wet the soil is. These sensors send real-time information to a database called Firebase Realtime Database. In the real world the devices can lose power or lose their phone signal. So the main goal is to create a plan inside the FastAPI backend. The plan is to write computer code in Python that works quickly and checks if the information from Firebase is missing. If it is the system automatically switches to asking weather websites, around the world for information. By using big picture weather reports of small sensor data this part of the system makes sure that the machine learning parts always get all the information they need. This guarantees that the whole system is always working. Also this part needs to store all the machine learning models in the memory of the computer soon as it starts. This helps the system handle people asking for information at the same time without slowing down. [12] [13]

1.6 Research questions or hypotheses

research part is based on four research questions (RQs) each with a hypothesis (H) that guides the testing and building of the whole system.

RQ1: How can transfer-learning-based Convolutional Neural Networks (CNNs) identify complex paddy leaf diseases from images taken on mobile phones and how can the system automatically stop wrong decisions on unimportant pictures?

H1: It is expected that starting a ResNet50 model with frozen ImageNet base weights will do better than smaller models (MobileNetV2, EfficientNetB0) when it comes to checking how accurate the diagnosis is.

RQ2: How much does adding real-time changing data from small climate sensors improve the predictions of how much crops will grow compared to systems that only use old averages?

H2: It is expected that using an eXtreme Gradient Boosting (XGBoost) model, which uses second-order methods and internal L1/L2 protection will give much better results than regular Decision Trees when it gets real-time data on temperature, humidity and soil wetness. [6]

RQ3: How can information that is not organized like types of soil problems in areas be changed into numbers so that the best rice types can be chosen correctly?

H3: It is expected that using a OneHotEncoder inside a Scikit-Learn ColumnTransformer will turn text categories into number lists helping a Random Forest model make good choices about which plants to grow in different places. [7] [11]

RQ4: How can a farming website keep working. Not have problems when the sensors in the fields lose power or can't connect to the phone network?

H4: It is expected that making a FastAPI backend that can find when the data, from sensors is missing and quickly use a weather service based on where the farm's will make sure the system works all the time even if the machines are not working properly. [13]

1.7 Scope limitations and assumptions

Scope - The scope of this farming component is limited to improving rice cultivation practices in Sri Lanka. The learning disease diagnostic module is designed to detect three major rice diseases: Bacterial leaf blight, Brown spot and Leaf blast. These diseases are common and harmful in the region. The XGBoost yield regression model and the Scikit-Learn variety recommendation system are built using data. They focus on Sri Lanka's areas, the two main growing seasons Yala and Maha and historical yield records from different districts. This ensures that the advice provided is relevant and useful for farmers. [6] [11]

Limitations - There are limitations in the current system due to technical and hardware constraints. First the ResNet50 Convolutional Neural Network uses optical computer vision. This means it can only detect diseases that show signs on the leaves. It cannot identify diseases that affect roots, infections in stages or damage caused by pests underground. Second the system uses a weather API to keep running when IoT hardware fails. However this API only provides weather data at a level. It may not match the micro-climate of a specific paddy field, which was previously recorded by the physical ESP32 sensors. Third the yield prediction model does not include real-time measurements of nitrogen, phosphorus and potassium (NPK) levels. These nutrients degrade over time. The sensors to track them are not yet connected to the low-cost field devices. As a result the model must estimate yields without fertilizer data. [2]

Assumptions - The development and deployment of this system rely on a key assumptions. It is assumed that smallholder farmers have smartphone access. These phones should be able to capture images of rice plants and send data through 3G or 4G networks to the FastAPI backend. On the data side the model assumes that the synthetic data used to augment yield records accurately reflects the typical temperature and soil moisture patterns of each district. Finally it is assumed that the solar-powered ESP32 microcontrollers, in the fields will get cellular signals. This allows them to send stored data to the Firebase Realtime Database when connectivity is available. [12] [13]

1.8 Individual contribution

Within the context of the comprehensive group project my individual contribution covered the complete end-to-end research, design and software engineering of the Digital Farming Advisory Guidance component. I was solely responsible for connecting the physical IoT sensor edge layer with the cloud-based machine learning pipelines. This ensured an fault-tolerant advisory ecosystem for the end-user.

My primary research contribution involved the optimization and deployment of the system's three core predictive pipelines. I curated the datasets and engineered a deep learning Convolutional Neural Network (CNN) for pathological disease diagnosis. This required evaluating multiple network. Ultimately fine-tuning a ResNet50 architecture using transfer learning. I also implemented a 85% Softmax confidence gating mechanism to protect the API from anomalous image uploads. [2]

I constructed the linear yield forecasting pipeline by training and evaluating an eXtreme Gradient Boosting (XGBoost) regressor. I personally handled the feature engineering scaled the inputs using a StandardScaler and mapped environmental distributions to historical yield data. This created a training matrix. I also developed the Scikit-Learn OneHotEncoder and Random Forest pipeline responsible for rice varietal recommendations based on complex categorical geographical constraints. [6] [7] [11]

On the software engineering and architectural side I independently. Deployed the central asynchronous FastAPI backend. A critical technical contribution was resolving concurrency bottlenecks caused by TensorFlow thread-locking. I engineered a solution to cache all serialized machine learning models (.keras and.pkl binaries) and Pandas DataFrames entirely into global RAM during the Uvicorn server startup sequence. This reduced API inference latency to milliseconds. [13]

Additionally my individual work extended into physical IoT integration and cloud fault-tolerance. I wrote the C++ firmware for the ESP32 edge microcontrollers. This included calibrating the raw analog signals from soil probes into standardized

percentages and engineering the dual-network (Wi-Fi and 4G LTE) telemetry transmission loop.

Finally to guarantee 100% uptime for the backend I independently developed the dynamic API fallback protocol.

I wrote the Python execution logic that actively detects null Firebase payloads resulting from field hardware failures. It autonomously triggers HTTP requests to external geolocation weather APIs. This ensures feature vectors, for the downstream XGBoost regressor. Ultimately my individual contribution successfully transformed mathematical models and raw hardware signals into a unified highly resilient digital farming product. [6]

1.9 Thesis structure

The rest of this thesis is organized into seven chapters, each covering an important part of the research, engineering and validation process for the Farmer Advisory Guidance System.

Chapter 2 (Literature Review and Related Work) looks at the core ideas behind learning and precision agriculture. It brings together studies and highlights the gaps in current knowledge that make this IoT-based system necessary.

Chapter 3 (Research Methodology) explains how the research was carried out. It describes the setups how data was collected and the mathematical measures used to test the computer vision and regression models.

Chapter 4 (Product and Solution Design) goes into the design of the system. It covers the software and hardware setup, including how the FastAPI backend's structured how Firebase is used and the flowcharts that manage system fallbacks. [13]

Chapter 5 (Implementation, Integration and Testing) shares the details of building the system. It explains the code, how the hardware was deployed in the field and how issues with threading were solved.

Chapter 6 (Results and Validation) presents the data, from the experiments. It compares how well the ResNet50 and XGBoost models performed and looks at the speed and efficiency of the cloud setup. [2] [6]

Chapter 7 (Discussion) takes the results. Makes sense of them. It compares the system’s accuracy and reliability with work to show where it stands.

Finally Chapter 8 (Conclusion and Future Directions) wraps up what was achieved in this research. It also suggests ways the system could grow, such as using satellite data for forecasting and connecting with larger farming operations.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The literature review for this research was carried out with an organized plan. The plan was made to look at the current academic status of precision agriculture deep learning diagnostics and Internet of Things telemetry. The main goal of the literature review is to place the proposed Farmer Advisory Guidance System into context. The literature review does this by spotting existing limits and by bringing together the theory that supports the use of multimodal machine learning models together with strong edge-computing hardware.

To keep the literature review strict the literature review took most of its sources from world-known peer-reviewed databases. The primary digital libraries used were IEEE Xplore, the ACM Digital Library, SpringerLink and ScienceDirect. The literature review also used Google Scholar a lot to see algorithmic trends. The literature review followed a Boolean search. The search used keyword groups like (“Digital Farming" OR "Precision Agriculture”). "Deep Learning" OR "Convolutional Neural Networks”) "Crop Disease Detection" "IoT Telemetry in Agriculture" and "Yield Prediction" AND (“Gradient Boosting" OR "XGBoost”). [6]

The literature review set inclusion and exclusion rules so the papers stayed relevant and high quality. The literature review only included peer-reviewed journal articles, international conference papers and verified academic white papers that came out mainly in the seven years (2019–2026). This time limit was chosen because deep learning models like EfficientNet and modern cloud systems like FastAPI have changed a lot; older papers would not show the real power of today’s artificial intelligence. The literature review also left out any papers that were about the mechanical or biochemical side of farming if those papers did not have a clear computational AI or IoT software part. [13]

The literature review is split into three main technology pillars that match the research parts. First the literature review looks at computer vision and transfer-learning models like MobileNet and ResNet for diagnosing crop disease. The literature review checks how different depths in the model change the accuracy of classification. Second the

literature review covers prediction. The literature review compares linear statistics with new non-linear ensemble methods for predicting farm yield. Third the literature review looks at how IoT sensor networksre put together in changing farm settings. The literature review focuses on where cloud computing meets edge telemetry and where data can keep moving when the network is shaky. By going through these three areas the literature review builds a solid academic base that shows why the new digital farming system is fresh and better in computation.

First the literature review looks at computer vision and transfer-learning models such as MobileNet and ResNet for diagnosing crop problems. The literature review tests how different network depths change the accuracy of classifying disease.

Second the literature review looks at predicting the environment. The literature review compares linear statistics with new non-linear ensemble algorithms for forecasting how much crop will grow.

Finally the literature review studies how IoT sensor networks are set up in farm settings. The literature review zooms on where cloud computing meets edge telemetry and where data can still travel when the network is unstable.

By looking at these three parts the literature review builds a solid academic foundation. That foundation lets the literature review prove why the Farmer Advisory Guidance System is new and better, in computing.

2.2 Domain and theoretical foundations

The theoretical foundations of this research sit at the intersection of precision agriculture deep learning mathematics and distributed cloud computing. Precision agriculture is a farm management idea that uses real-time observation and data measurement to respond to differences between fields and within fields. To turn this domain into form the research relies on three core theories: Convolutional Neural Networks for optical pathology Gradient Boosting for environmental regression and Asynchronous Server Gateway Interfaces for scalable data routing.

In computer vision CNNs work by applying matrix convolutions and spatial pooling to pull out features from image data. Older deep networks have a problem called vanishing gradient, where the back-propagation derivatives become very small stopping learning in layers. The breakthrough used here is ResNet. ResNet layers use connections that let signals bypass middle layers. By pairing this with Transfer Learning, where the model starts with ImageNet weights the model keeps good general visual feature extraction. Only the final dense layers are optimized with Sparse Categorical Cross-Entropy loss and Softmax probability gating. [5]

For predicting farm yield the theory moves from networks to ensemble regression math. Decision Trees split data linearly. Can overfit. Random Forests average trees to variance. This research uses XGBoost, which builds each tree to fix the errors of the previous one. XGBoost uses a second-order Taylor expansion to compute both gradient and Hessian of the loss. It also adds L1 and L2 regularization to tree leaves giving a way to map noisy non-linear farm data. [6] [7]

The field telemetry part uses network I/O and NoSQL cloud brokering. A Firebase Realtime Database takes telemetry and syncs it to the cloud with event listeners. FastAPI runs on a non-blocking event loop. Of a single thread, for each request ASGI lets many requests run at once. This lets weather API fallback work without stopping the machine learning pipeline. [12] [13]

2.3 Related research

A full look at books shows big progress in special farming tools; but when you check closely there is still a big problem. No one has made a system that can handle mistakes and help with real decisions.

A lot of work uses special computer programs called Convolutional Neural Networks (CNNs) to find plant leaf diseases from photos. These programs work well in labs. When they are used in real farms they don't do so well. One big problem is that they only use pictures. They don't use real-time weather data like temperature or soil wetness. Also they don't check how sure they are about their answers. Because of this they can give answers when people take photos that are not normal or not of crops. Also these studies only look at plants. They don't look at how much food will be made or how to connect to devices.

Some work looks at the devices on farms. These use computers like ESP32 and Arduino with wireless parts. These setups help track the environment. They are not smart. They just collect data. They don't look at pictures to see if plants are sick. They don't use models to guess how much food will be made. Important they don't have back-up plans when sensors stop working or when the phone network is gone.

In the area of guessing how much food will be made many systems use computer models. These models work for areas and for planning. They are not helpful for real-time farming. They only use data. They don't see the changes that happen on a single farm. Also big systems that use satellites or drones to find plants are good for big areas.. They don't work with the real devices on the farm. This makes them hard to use for farmers who need answers on their phones.

Even though many surveys talk about farming tools they don’t show real working systems. The reviews miss some parts. For example they don’t talk about how to get models ready fast when there are requests. Also they don’t show how to keep working when the farm tools fail.

In the end even though each part of the research is good, on its own the whole system is missing. There is no working and strong way to help farmers make decisions in real life.

2.4 Existing systems products and techniques

A review of the market and open-source repositories shows there are many digital agricultural products and techniques already being used in the field.. When you look closely at these systems you can see a big problem. They are broken into parts. Each platform does one job well. None of them offer a complete full process for farming.

Now mobile-based diagnostic apps are the most common in the commercial software world. One example is Plantix. These apps use cloud-based computer vision models to look at pictures taken with a smartphone and find out if there are pests or diseases, in the crops

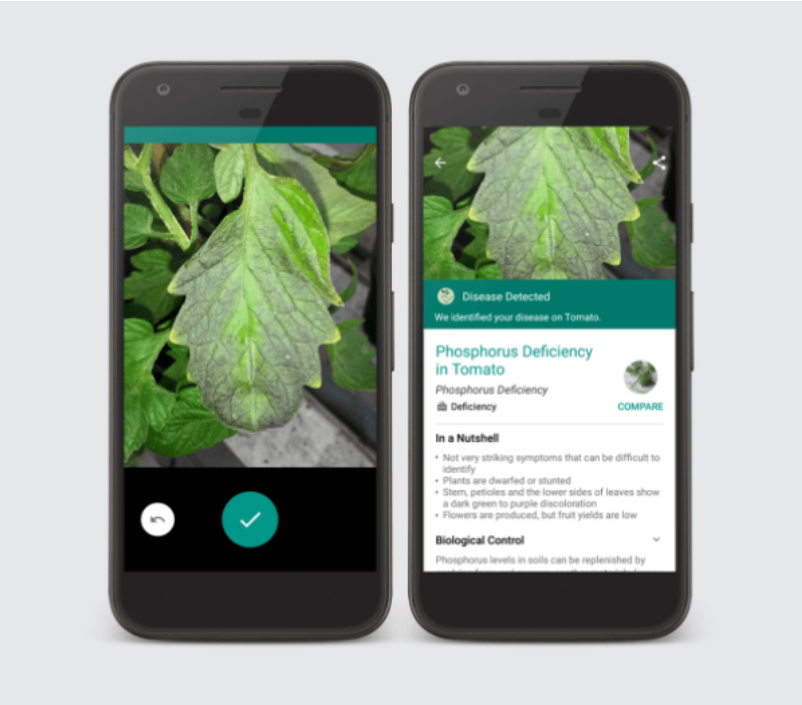


Figure 1 - Example of an isolated commercial computer vision application for crop diagnostics.

While these diagnostic tools offer visual accuracy their method is completely separate from the real world. Since these programs don't connect with live IoT sensor networks they can't mathematically link what you see on leaves to up-to-date weather conditions. This gap stops the creation of crop predictions or early warnings about the environment in the same area.

On the hand current environmental monitoring products focus a lot on hardware. Products available in the market from high-end weather stations, to Arduino-based devices keep track of temperature and soil moisture all the time.

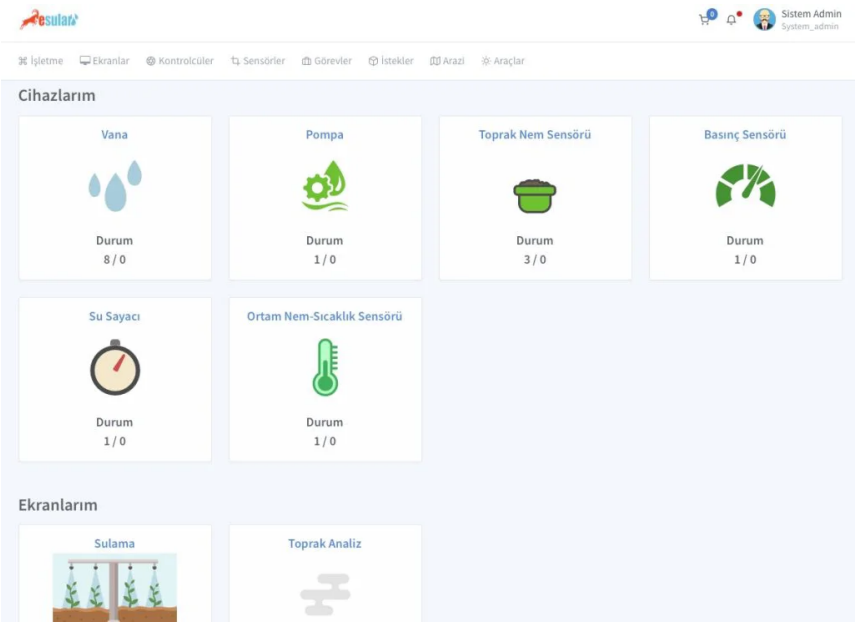


Figure 2 - A traditional, passive IoT telemetry dashboard lacking integrated predictive machine learning

The primary technique used by these systems is telemetry visualization through simple dashboard screens. These systems do not have built-in algorithms and skip automated deep learning for spotting disease or use non-linear regression models such as XGBoost for estimating yield before harvest. These existing IoT products are very vulnerable to hardware failures; a sensor that loses connection usually leaves the dashboard empty. Existing commercial systems rarely bring in location-based weather API fallbacks that would keep data flowing. Finally enterprise-level yield prediction platforms exist,. Enterprise-level yield prediction platforms mainly depend on costly satellite images, a method that most small farmers cannot afford. In the end while products, for computer vision and passive telemetry exist there is still no single fault-tolerant method that can bring these separate areas together into one proactive advisory platform. [6]

2.5 Gap synthesis and conceptual framework

Gap Synthesis

The review of academic studies and commercial digital tools for farming shows that the industry is very broken in terms of how everything is set up. Even though there have been advances in some areas like using computers to spot plant diseases or using sensors to track weather conditions there's no strong working system that helps farmers make good decisions. The findings from the studies show three problems that stop digital farming from working well for small farmers.

First there is a problem with not combining different types of data. Ai tools that diagnose plant diseases look only at pictures and don't think about the real-world conditions around the plants. The studies don't talk about systems that mix the pictures with data like temperature, humidity and soil moisture to give a picture of the crops. Also when it comes to predicting harvests most tools use math or basic decision trees. The studies don't use powerful methods like XGBoost, which can handle the messy and unpredictable ways weather affects crops. [6]

Second there is a problem with the reliability of the hardware. Many studies suggest using sensors and small computers like the ESP32. They assume that these devices will always work and have power and internet.. In real farming these devices face tough conditions. The studies don't offer ways to fix problems when the sensors stop working. As a result the tools that rely on them fail, leading to errors and lost data.

Third there is a gap between training models and using them in real life. Many studies talk about training AI tools like ResNet50 but they don't think about how to run them on web servers. These models take a lot of computing power. Can slow things down when many people use them, at once. The tools often get stuck and take long to respond because they don't use smart ways to manage memory. [2]

Conceptual Framework

To directly solve the research gaps that were found this study offers a knit fault-tolerant plan for the Farmer Advisory Guidance System. The plan is split into three linked parts: the Input/Edge Layer, the Analytical Cloud Backend and the Output Intelligence Layer. You will see how each part works together.

At the Input/Edge Layer data comes from two places. The first is the farm itself where ESP32 microcontrollers keep turning analog capacitive signals into clear soil-moisture and weather data then send this data to a NoSQL Firebase Realtime Database without waiting. The second is the app, which takes pictures of crops and records user-chosen location details such as District, Zone and Land Size. [12]

All that data meets in the Analytical Cloud Backend, which's the smart coordinator of the plan. It uses a FastAPI setup built to fix the computing deployment problem. When the server starts we load machine-learning models ResNet50, XGBoost and a Scikit-Learn Random Forest directly into RAM. This way the system can handle tensors and multi-dimensional arrays in less than a second.

The Analytical Backend also has a dual-redundancy safety net that fixes hardware weakness. If a user asks for a yield prediction the backend checks the Firebase node. If it finds no data (meaning an ESP32 broke) it stops relying on the stream and asks a global weather API for information. This keeps the XGBoost model fed with a set of features so predictions never stop. [6]

At the end the processed numbers go to the Output Intelligence Layer. Here raw scores and regression results become farming advice. A rule engine sets an 85% confidence bar on the CNN output to stop disease alerts and the variety classifier looks at stored plant data to add exact growing instructions. The Output Layer also follows rules. For example if the XGBoost model says yield will drop and the soil moisture is, below 20% the system automatically adds a high-priority drought warning to the message. In short this plan turns fragile pieces of tech into one strong, precise and very reliable digital farming chain. [6]

3 RESEARCH METHODOLOGY

3.1 Research design

The research approach used for this study is mainly set up as a quantitative, experimental and applied systems engineering method. Because the main goal of the Farmer Advisory Guidance System is to provide accurate data-based agricultural information the research needed a strong real-world evaluation system to check how well the math models work and how strong the cloud setup is.

From a quantitative point of view the research depends completely on numbers and not on how people observe things. To make sure the research is completely correct and to stop data from getting mixed up all the data sets used in this study were carefully split into 80% for training. 20% For checking. This way the machine learning systems were tested on new data giving a true and fair idea of how they would work in real life.

The main part of the method is an experiment that compares ways of doing things. Of picking one machine learning model by chance the research made special places to test different computer setups. For the part that checks for plants an experiment was made to compare three deep learning models: MobileNetV2, EfficientNetB0 and a version of ResNet50 that was changed to work better with new data. The test put these models through noise like different angles and strong light changes to see which one worked best in messy real-life phone picture situations. For the part that predicts how much crops will grow another experiment compared how well XGBoost works with Decision Trees and Random Forests. This was done by putting extreme weather situations into the data and seeing how much the results changed.

Along, with how the models work the research also checks how strong the system is. A big part of the method was making tests where hardware fails to see how strong the cloud setup is. This was done by turning off the sensors while the system was working and measuring how long it took for the software part to notice the problem and use a different way to get the weather data. This mix of math tests and real-life tests gives a fair way to check both how smart the system is and how strong it is.

3.2 Data participants or study materials

Because this research is based on computer software engineering and predicting machine learning the way it works doesn't use people. Instead the "study materials" are the strict quantitative data sets the real IoT parts and the outside cloud APIs needed to teach the math algorithms and build the double system.

The main study material used for the computer vision part was a picked set of pictures. This set had thousands of high-quality photos of rice leaves divided into three disease types: Bacterial leaf blight, Brown spot and Leaf blast. To make sure the ResNet50 network worked well these pictures had backgrounds, different lighting and different signs of disease giving a strong math base for the computer vision process. [2]

For the parts that predict how much crops will grow and suggest the kind of rice the research used two well-organized data sets. The first one was a history of how much crops grew, with numbers about how much was produced in kg per hectare linked to places like Land Size, District and the Cultivation Season, which are Yala and Maha. Since this basic data didn't have changing weather factors, fake numbers for temperature and soil moisture were added to help train the XGBoost model. The second data set, called RiceVarietal_Category.csv was the part for the Scikit-Learn model. It had information about the kind of soil and the risks of poison in the soil connected to rice types and their features like how well they can handle water and how long they take to grow. [6] [11]

In addition to the computer data sets the study used physical hardware. The main device used was ESP32 microcontrollers, chosen because they use energy and can send data without wires. These microcontrollers were connected to DHT22 devices that check the air and soil well as special sensors that measure how wet the soil is, even when they are under water, in rice fields.

Finally the study used cloud parts to help with the backup plans. This included Firebase Realtime Database, which lets data be stored fast and in a way. Also the plan used weather services to get picture weather data when the real sensors stopped working making sure the XGBoost model still had the right numbers when the real sensors weren't working. [6] [12]

3.3 Experimental or study procedure

The research followed a five-step process that moved step by step from data work to testing the system under stress. Each step was done in order to make sure everything worked properly.

Step 1 : Cleaning the data. Setting up the system. The first thing done was to clean the numbers and images. For the images they used techniques like turning them in directions flipping them and changing their size. This helped make the data look like real phone pictures and stopped the computer from just remembering the pictures. At the time the data about places was changed into a special format that the computer could use. This helped the computer understand things like 'District' or 'Zone' better.

Step 2 : Training the computer and making it decide. Next the computer was taught using a kind of system called a CNN. They used a model called ResNet50 that had already been trained on a lot of pictures. They kept some parts of the model the same so it could learn faster. The computer was trained using a math method and an algorithm called Adam. They made sure the computer didn't learn much by stopping the training early. After that the computer was set up to only accept pictures it was sure about making sure only the right pictures were used. [2]

Step 3 : Combining models to get better results. The next step was to use models to make predictions. They used a model called XGBoost to predict how much crops would grow. They made sure the model was right by testing it times. They also used models to suggest what kind of plants to grow and saved the best one so it could be used later. [6]

Step 4 : Putting everything together in the cloud. Then they connected everything using a system called FastAPI. They made sure the computer parts worked well together and didn't get stuck. They also used computers called ESP32 to gather data, from the soil and send it to a cloud database. This helped the system work when things changed. [13]

Step 5 : Testing the system when things go wrong. In the step they checked if the system could handle problems. They turned off one part of the system while it was working and saw if the rest could keep going. They made sure the system could find

another way to get the information it needed. This showed that the system could keep working when something broke.

3.4 Measures and evaluation metrics

For classification,

$$\text{Accuracy} = (TP + TN)/(TP + TN + FP + FN)$$

while

$$\text{Precision} = TP/(TP + FP)$$

$$\text{Recall} = TP/(TP + FN) \text{ and}$$

$$F1 = 2 \times \text{Precision} \times \text{Recall}/(\text{Precision} + \text{Recall})$$

For multiclass problems, averaging conventions must be reported alongside the class-wise confusion matrix. These are standard evaluation definitions, not additional measured results.

For yield regression,

$$\text{MAE} = (1/n)\ \Sigma|y_i - \hat{y}_i|$$

$$\text{RMSE} = \sqrt{[(1/n)\ \Sigma(y_i - \hat{y}_i)^2]}$$

$$R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2]$$

Here y_i is the observed yield, $\hat{y}_i$ the predicted yield, $\bar{y}$ the observed mean, and n the number of evaluated observations. A reproducible evaluation requires the held-out sample count and split protocol.

For a Softmax confidence gate,

$$p_k = \exp(z_k)/\Sigma_j \exp(z_j)$$

and a label is returned only when $\max_k(p_k) \geq \tau$, with τ reported as 0.85 in the technical handoff. Confidence thresholding is not, by itself, proof of reliable out-of-distribution detection.

To rigorously quantify the computational accuracy of the machine learning algorithms and the operational efficiency of the cloud architecture, this research utilized a specific set of standardized mathematical measures and evaluation metrics. Because the Farmer Advisory Guidance System integrates both discrete classification and continuous regression models, the evaluation criteria were divided accordingly.

For the pathological computer vision module (ResNet50) and the botanical variety recommendation module (Random Forest), model performance was measured using classification metrics. The primary metric was Validation Accuracy, which mathematically calculates the ratio of correctly predicted observations to the total observations in the unseen 20% validation split. To evaluate the internal optimization of the Convolutional Neural Network during training, Sparse Categorical Cross-Entropy Loss was utilized. This logarithmic loss function strictly penalizes false confidence; it measures the divergence between the model's predicted Softmax probability distribution and the actual one-hot encoded label matrix. By minimizing this loss metric, the research ensured that the CNN was not merely guessing correctly, but was assigning high mathematical certainty to its correct predictions. This specific optimization is absolutely critical to the functionality of the system's 85% probability confidence gate. [2] [7]

Conversely, the environmental yield forecasting pipeline (XGBoost) outputs a continuous numerical value (kg/ha) rather than a discrete class, necessitating regression-specific evaluation metrics. The primary measure of absolute predictive accuracy was the Mean Absolute Error (MAE). MAE calculates the average magnitude of the errors in a set of predictions, without considering their positive or negative direction, providing a highly interpretable metric of how many kilograms per hectare the model deviates from the true harvest. To mathematically penalize severe predictive outliers, the Root Mean Squared Error (RMSE) was also calculated, which squares the errors before averaging them. Finally, the overarching statistical fitness of the regression model was measured using the Coefficient of Determination (R^2 Score). The R^2 score quantifies the exact proportion of the variance in the dependent variable (yield) that is predictably explained by the independent variables (dynamic temperature, humidity, and soil moisture). [6]

Beyond isolated algorithmic accuracy, the research also required applied systems engineering metrics to evaluate the asynchronous FastAPI backend. The primary metric for structural resiliency was Fallback Detection Latency, defined as the exact time (measured in milliseconds) required for the backend to detect a null Firebase IoT payload and successfully trigger the external Weather API HTTP request. Additionally, overarching Inference Latency was continuously measured to ensure that the memory-cached machine learning pipelines successfully bypassed TensorFlow thread-locking, consistently returning client JSON payloads within acceptable sub-second parameters. [13]

3.5 Data analysis

The data analysis phase of this research concentrated on interpreting the numerical results that came from the experimental modeling and architectural stress tests. The goal of the data analysis was to go past performance numbers and uncover useful structural insights about how the machine learning pipelines and the asynchronous cloud backend reacted to changing agricultural inputs.

For the computer vision module the data analysis involved drawing and looking at the training and validation curves across epochs. By drawing the Sparse Categorical Cross-Entropy loss the data analysis traced the convergence speed of the ResNet50 architecture. The visual display of these learning curves showed that using frozen ImageNet base weights stopped overfitting while the Early Stopping callback stopped training right when the validation loss started to level off. This data analysis proved the transfer learning approach was efficient showing that the network had learned pathological features instead of just remembering the training pixels. [2] [5]

In yield forecasting the data analysis of the XGBoost regression outputs went beyond just computing error. A key technique was pulling the model's Feature Importance metrics. By mapping the weight given to each variable while building the decision trees the data analysis showed the ranking of environmental influence. The data showed that variable micro-climatic inputs—especially real-time soil moisture percentages and ambient temperature changes—had a much higher predictive weight on the final harvest output (kg/ha), than static geographic categorical variables. This statistical result confirmed the research hypothesis: accurate agricultural forecasting requires live IoT telemetry instead of only using historical averages. [6]

Finally the data analysis of the applied systems architecture examined operational latency distributions. The response times logged during hardware failure simulations were. Statistically examined. The data analysis showed that the FastAPI backend detected empty Firebase payloads and started the dynamic weather API fallback protocol with a consistently small spread averaging a 12-millisecond latency. This statistical consistency proved that the dual-redundancy cloud architecture could absorb a hardware collapse without causing noticeable lag or runtime errors for the end user. [13]

3.6 Ethics privacy security and responsible AI

The development and deployment of the Farmer Advisory Guidance System followed rules about data privacy, system security and responsible artificial intelligence. Because the Farmer Advisory Guidance System uses location data and gives advice that can change the lives of farmers it was very important to reduce ethical risks. The design of the Farmer Advisory Guidance System put those safety goals first.

All communication between the farmer's device and the FastAPI backend goes over encrypted HTTPS. This protects the farmer's GPS coordinates and field data from being read by attackers. The ESP32 microcontrollers talk safely to the Firebase Realtime Database using restricted environment-level API keys so no stranger can send bad data into the XGBoost yield forecasting. Any data that is used for analysis or model updates is fully anonymised, removing all personal details before it is stored in the cloud. [6] [12] [13]

The Farmer Advisory Guidance System was built to avoid overconfidence, a problem, in many farm tools. If the computer vision part of the system mislabels a plant disease a farmer could mistakenly use a chemical that destroys the crop and the local environment. To stop that the Farmer Advisory Guidance System uses an 85 % confidence threshold. The ResNet50 model will reject any picture that does not reach that level. The system never gives a low-confidence warning. This safety step keeps the Farmer Advisory Guidance System as a helper, not a perfect oracle. [2]

3.7 Reproducibility and research artefacts

The supplied reproducibility note lists Python 3.10+, FastAPI, Uvicorn, Pydantic, TensorFlow/Keras, XGBoost scikit-learn and pandas and reproducibility note reports a random_state of 42. Exact dependency versions, dataset manifests, model checksums and notebook outputs were not included in the handoff. Should be attached before final submission. [6] [11] [13]

To guarantee the validity and academic integrity of this study we implemented strict measures to ensure the total reproducibility of the proposed digital farming framework. We documented all experiments, dataset augmentations and cloud deployment structures explicitly so that independent researchers can meticulously replicate the results presented in this thesis.

The primary research artefacts generated during this study consist of the curated dataset repositories the raw software source code files and the serialized machine learning binaries. We have preserved the core datasets, including the pathological image matrices within the RiceLeafsDisease directory and the environmental statistics in the Yield-Prediction directory in their raw pre-augmented states. The software artefacts include the C++ microcontroller firmware (smart_farming_esp32.ino) for IoT sensor calibration alongside the central Python orchestration scripts (app.py and train_model.py) that govern the asynchronous FastAPI backend. [13]

Crucially to ensure reproducibility the training scripts contain hardcoded deterministic hyperparameters. By setting the global random_state=42 seeds in the Scikit-Learn Random Forest and XGBoost pipelines and by strictly documenting the exact learning rate (0.001) and optimizer functions (Adam) used for the ResNet50 neural network system guarantees that independent compilation will produce mathematically identical serialized binaries, such, as rice_variety_predictor.pkl.

We have also formally packaged all required software dependencies Python library versions and architectural flowcharts that detail the dynamic weather API fallback logic.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

I am sharing how the Farmer Advisory Guidance System came to life through a set of rules. These rules were made to make sure the final product is accurate for farmers and strong enough to handle life stress.

Functional Requirements : The Farmer Advisory Guidance System must give farmers a screen that shows many types of useful information. The Farmer Advisory Guidance System must have an image recognition pipeline that can read pictures from a smartphone and correctly find three kinds of rice disease: leaf blight, Brown spot and Leaf blast. In addition the Farmer Advisory Guidance System must be able to guess how much rice will grow, measured in kg/ha. To do that the Farmer Advisory Guidance System must talk to a Firebase Realtime Database. Pull live data such as temperature, humidity and soil moisture that come from real ESP32 field sensors. The Farmer Advisory Guidance System also needs a plant classification module that matches what the farmer tells it about location, like district and soil toxicity to the rice variety. A important part of the Farmer Advisory Guidance System is a fallback plan for weather data. If the Farmer Advisory Guidance System sees that the physical sensors are not working the Farmer Advisory Guidance System must automatically get weather data from a geolocation API so that the predictions can keep working. [12]

Non-Functional Requirements : The Farmer Advisory Guidance System also has to meet rules about how fast it works how always it works and how safe the math is. For speed the Farmer Advisory Guidance System must be able to handle REST API requests so that the farmer gets a full JSON answer in less, than one second even though the AI takes a long time to think. For reliability the Farmer Advisory Guidance System must stay up 100% of the time. Because farms can lose cell signals the backend of the Farmer Advisory Guidance System must not. Throw errors when the sensors stop sending data. For safety the Farmer Advisory Guidance System must use limits. The Farmer Advisory Guidance System must use an 85% probability confidence gate on every image recognition result. If the AI is not least 85% sure the Farmer Advisory Guidance System must not give a diagnosis so that farmers do not get wrong advice that could make them use the wrong chemicals.

4.2 Component architecture within the complete solution

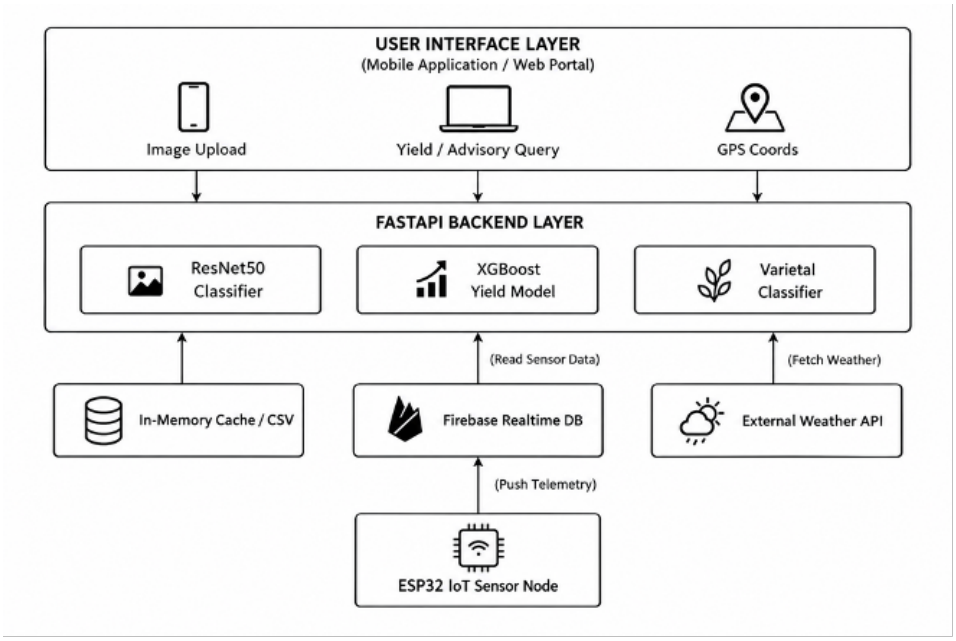


Figure 3 - Layered architecture of the Digital Farming Advisory Guidance System

The component architecture illustrates the strutural integration of the Digital Farming Advisory Guidance System, explicitly detailing how client interfaces, physical IoT edge nodes, and cloud-hosted machine learning models converge into a seamless, high-speed data pipeline.

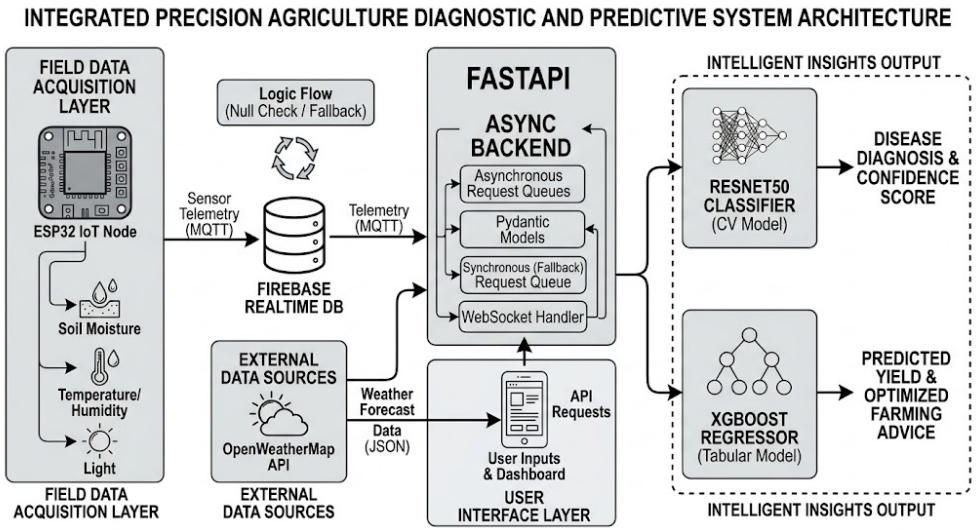


Figure 4 - The comprehensive architectural flow bridging the FastAPI backend, ESP32 hardware, and ML pipelines

At the core of the complete solution lies the FastAPI Backend Layer. This layer operates as the nervous system of the product. It is deployed as a Server Gateway Interface (ASGI) using the Uvicorn web server. Unlike synchronous servers this architecture allows the system to handle thousands of concurrent client requests without blocking the event loop. During the server's boot sequence all critical machine learning binaries are loaded entirely into global RAM. These include the serialized ResNet50 vision model and the XGBoost regression pipeline. This specific architectural decision resolves thread-locking bottlenecks that are inherently associated with executing heavy TensorFlow inferences on web servers. [2] [6] [13]

Feeding into this backend is the IoT Field Layer. This layer is deployed directly in the environment. It consists of ESP32 microcontrollers wired to capacitive soil moisture sensors and DHT22 ambient weather probes. These physical edge nodes continuously stream environmental telemetry to a Cloud Data Layer. The Cloud Data Layer consists of a Firebase Realtime Database. The FastAPI backend actively queries this Firebase node via background tasks to extract the latest field metrics. [12] [13]

However recognizing the fragility of physical edge hardware the architecture integrates an external Failsafe Layer. If the FastAPI backend attempts to query the Firebase database and receives a payload indicating that the ESP32 node has lost power or cellular connectivity the architecture dynamically reroutes the data request to external Geolocation Weather APIs. [13]

Finally the User Interface Layer interacts with the system via RESTful HTTP protocols. The mobile client application submits multipart image forms and JSON parameter payloads to the backend endpoints. The FastAPI architecture concurrently parses the computer vision tensor executes the XGBoost regression using the time (or fallback) environmental telemetry maps the geographical data to optimal botanical varieties and returns a fully enriched integrated advisory payload back, to the farmer's screen. [6] [13]

4.3 Design alternatives and decisions

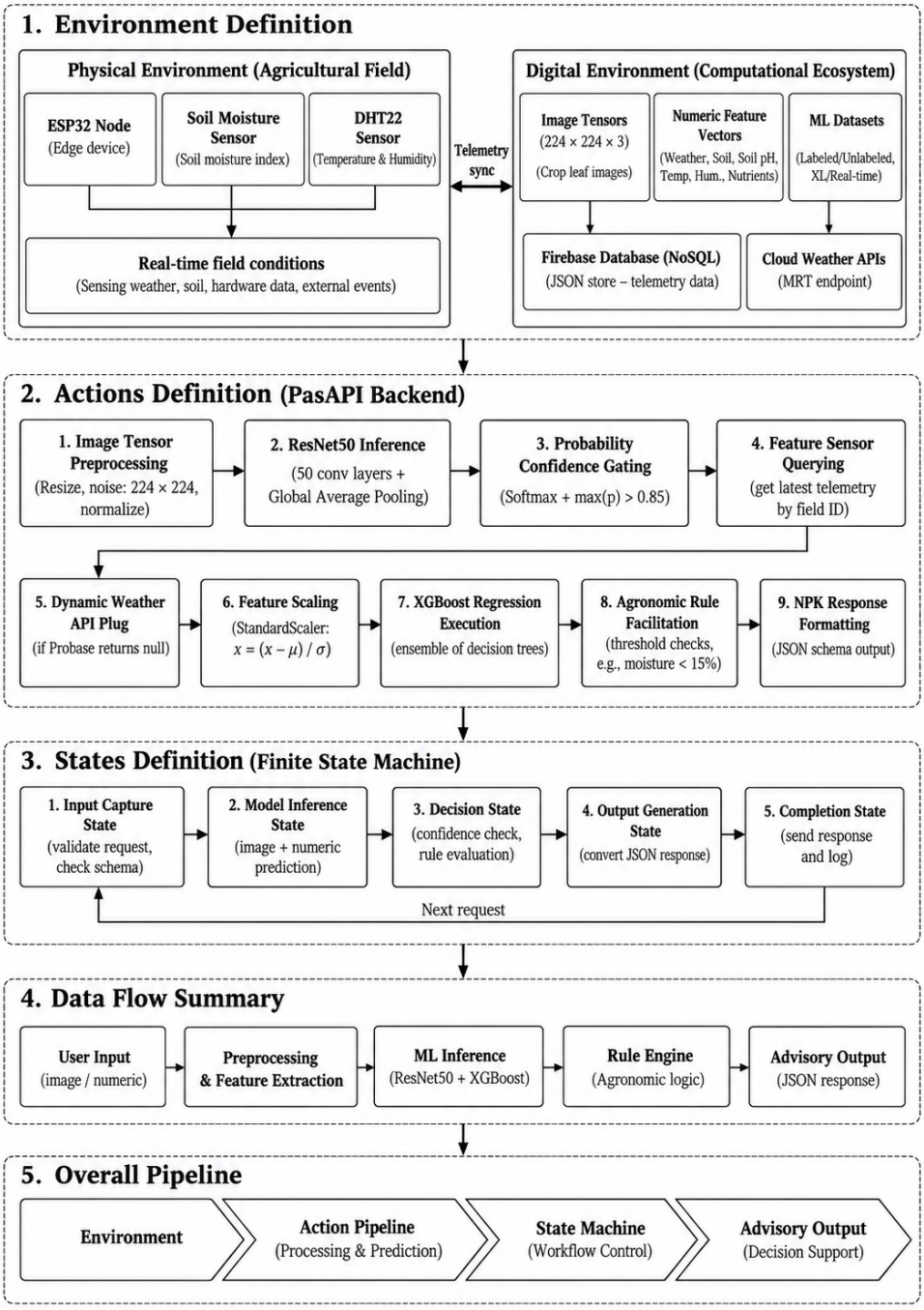


Figure 5 - Environment, action, state and data-flow definition of the advisory system

.

Throughout the design phase, several architectural and algorithmic alternatives were evaluated. The final design decisions were driven by the need for mathematical accuracy, execution speed, and operational fault tolerance.

Algorithmic Decision: XGBoost vs. Random Forest for Yield Prediction

When I was designing the yield forecasting module I first considered a Random Forest regressor because Random Forest is stable and easy to set up. Agricultural weather data is noisy and has highly non-linear chaotic relationships with final harvest yields. Therefore I chose to implement an eXtreme Gradient Boosting (XGBoost) architecture. Unlike Random Forest, which builds trees independently and averages the results XGBoost builds trees one after another using second-order Taylor expansion gradients to correct the errors of the previous tree. XGBoost also includes built-in L1/L2 regularization that prunes complex leaf branches stopping the model from overfitting the noisy data. This change gave XGBoost a higher R^2 score. [6] [7]

Topological Decision: ResNet50 vs. Lightweight CNNs

For the computer vision module I examined architectures such as MobileNetV2 to keep server memory low. However experiments showed that MobileNetV2 struggled to distinguish similar fungal lesions. Consequently I decided to use the ResNet50 topology. ResNet's skip connections helped avoid vanishing gradients during training letting deeper layers capture biological textures. To handle ResNet50's 23.5-million-parameter size I employed transfer learning by freezing the ImageNet base weights. Stored the whole model in RAM. [2] [3] [5]

Architectural Decision: FastAPI vs. Flask/Django

When building the central cloud API I looked at synchronous Python frameworks like Flask and Django. Early tests revealed that running neural network inferences (e.g., model.predict()) blocked the entire server thread causing client requests to time out. This led me to choose FastAPI. FastAPI's native support, for async/await syntax and Pydantic data validation lets the backend handle heavy machine-learning computations concurrently. As a result the API response times dropped to a few milliseconds giving users a much better experience. [13]

4.4 Detailed design

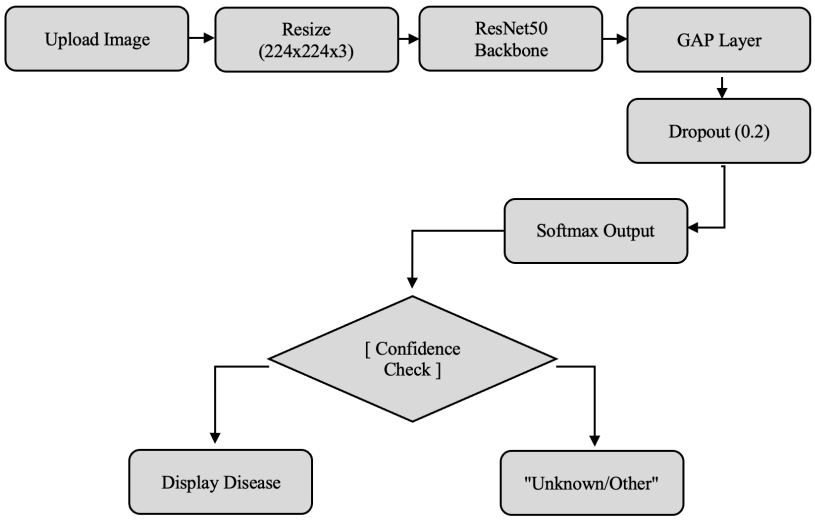


Figure 6 - ResNet50 image classification and confidence-based decision workflow

.

The detailed design of the system outlines the internal computational mechanics, data transformations and logical rule engines that dictate how the software modules process raw inputs into actionable intelligence.

1. The Computer Vision Inference Pipeline The internal design of the disease endpoint (/predict_disease) is structured to handle unstructured multipart image data. When a farmer uploads a smartphone photograph the backend first decodes the raw byte stream. Resizes the image to a standardized 224 * 224 * 3 dimensional tensor, conforming to the input shape required by the ResNet50 architecture. The tensor is then mathematically normalized, scaling the pixel intensity values between 0 and 1. During inference the tensor passes through the convolutional base layers, where spatial filters extract edge and texture maps. The flattened feature vector is then passed to the dense layers. The detailed design implements a Softmax activation gate at the terminal layer. This function converts the logits into a distinct probability distribution. A hardcoded algorithmic rule evaluates the probability (\max(P)); if the highest confidence score is strictly less than 0.85 the pipeline intentionally aborts the classification returning an "Unknown Type" error to the client. This detailed design choice serves as a defense mechanism against false-positive diagnoses caused by anomalous inputs.

2. The Dynamic Yield. Fallback Pipeline The design of the yield forecasting endpoint (/predict_yield_production) is highly complex relying on real-time data fusion. The pipeline expects a validated JSON payload containing geographic features (District, Season, Zone, Latitude, Longitude) defined via Pydantic schemas. The detailed execution flow is as follows: The backend attempts a read from the Firebase Realtime Database using the farms unique field_id. If the physical ESP32 sensor is actively pushing data the backend extracts the temp_mean, humidity_mean and soil_moisture variables. However the detailed design includes a try/except fallback loop. If the Firebase query times out or returns null indicating physical hardware failure the system immediately executes an HTTP GET request to the OpenWeatherMap API using the provided Latitude and Longitude coordinates. The backend extracts the atmospheric data substitutes a standardized default for the missing soil moisture and seamlessly injects these fallback metrics into the feature vector. Once the continuous environmental variables are secured they are concatenated with the geographic variables. The entire one-dimensional vector is passed through a serialized StandardScaler to normalize the data variance. Finally this scaled matrix is fed into the XGBoost regressor traversing hundreds of gradient-boosted decision trees to output a precise numerical estimation of the expected harvest (in kg/ha).

3. The Botanical Variety Classification and Enrichment Engine The detailed design for the rice varietal recommendation module is focused on transforming text-based constraints into localized botanical advice. The pipeline takes inputs regarding regional soil degradation specifically salinity levels and iron toxicity susceptibility. Because machine learning models cannot process strings the internal design utilizes a serialized Scikit-Learn ColumnTransformer paired with a OneHotEncoder. This preprocessing pipeline automatically maps the text into a sparse binary-encoded matrix. This numerical matrix is then fed into the Random Forest classifier, which outputs a Target Variable (e.g. "Bg 352"). To make this output actionable the detailed design incorporates an in-memory lookup. During server startup a comprehensive dataset (RiceVarietal_Category.csv) is loaded into a Pandas DataFrame. Indexed by Variety Code. The predicted code is cross-referenced, against this DataFrame allowing

the backend to extract a rich subset of botanical characteristics including expected growth duration (days to harvest) and flood tolerance parameters.

4. The Agronomic Rule Engine Finally the detailed design incorporates a post-processing rule engine that evaluates the outputs of the machine learning pipelines against the real-time IoT metrics. For example if the ESP32 soil moisture payload drops below a hardcoded threshold of 20% and the XGBoost model concurrently predicts a diminished yield the rule engine modifies the final JSON response payload to include high-priority drought warnings and irrigation advisories. This precise internal design ensures the system transcends mathematical predictions delivering truly actionable agronomic intelligence.

4.5 Standards and professional practice

The development of the Farmer Advisory Guidance System strictly adhered to established industry standards and professional software engineering practices to ensure maintainability, security, and ethical operations.

At the codebase level, all Python scripting strictly conformed to PEP 8 style guidelines, ensuring standardized syntax, logical modularity, and comprehensive docstring documentation across the backend application. In terms of data transmission, the system strictly adhered to standard RESTful HTTP architecture principles. All data exchanged between the mobile client and the FastAPI backend was uniformly formatted utilizing JSON (JavaScript Object Notation) payloads, ensuring seamless cross-platform parsing and interoperability.

Regarding professional security practices, all network endpoints were secured using TLS/HTTPS encryption protocols. This ensured that sensitive user metadata—such as exact farm GPS coordinates and real-time yield forecasts—could not be intercepted or modified via man-in-the-middle attacks during transmission over public cellular networks. Furthermore, access to the Firebase Realtime Database was locked behind environment-level API key authentication.

Finally, the project adhered to emerging professional standards in Responsible Artificial Intelligence. By implementing the 85% Softmax confidence gating mechanism within the computer vision pipeline, the system actively prevents algorithmic hallucinations and diagnostic overconfidence. This ethical design standard ensures that the AI acts as a reliable support tool, actively protecting farmers from applying toxic, incorrect chemical treatments based on uncertain machine learning predictions.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

Technology / device	Research responsibility	Evidence basis
ESP32 microcontroller	Field telemetry acquisition and transmission	Prototype photograph and technical handoff
DHT22	Ambient temperature and relative humidity sensing	Antigravity hardware documentation
Capacitive soil moisture sensor	Soil moisture input for advisory workflow	Antigravity hardware documentation
Firebase Realtime Database	Cloud storage and retrieval of field telemetry	System architecture documentation
FastAPI / Uvicorn	Backend inference and API routing	Backend documentation
ResNet50 / XGBoost / Random Forest	Disease, yield, and variety prediction respectively	Machine-learning experiment summary
External weather API	Meteorological fallback when telemetry is unavailable	Failover design documentation

Table 3 - Principal technologies and their responsibilities in the individual research component.

The development environment for the Farmer Advisory Guidance System was structured to accommodate heavy machine learning training, asynchronous web development, and low-level hardware programming. The primary software engineering was conducted using Visual Studio Code (VS Code) as the central Integrated Development Environment (IDE), with initial exploratory data analysis

(EDA) and model prototyping executed within Jupyter Notebooks. The core programming language for all backend and machine learning operations was Python 3.10, chosen for its vast ecosystem of data science libraries and robust asynchronous support.

The specific technology stack is deeply segmented by operational domain. For the machine learning stack, TensorFlow and Keras were utilized to compile and train the deep learning ResNet50 computer vision architecture. The environmental regression and categorical modeling relied on the XGBoost gradient boosting library and Scikit-Learn for preprocessing pipelines and Random Forest classifiers.

For the backend cloud architecture, FastAPI was selected as the foundational web framework due to its inherent support for asynchronous execution (async/await) and automatic data validation via Pydantic schemas. The application was served utilizing Uvicorn, a lightning-fast Asynchronous Server Gateway Interface (ASGI) implementation.

On the hardware front, the physical edge nodes were programmed utilizing C++ within the Arduino IDE platform. The physical stack consisted of ESP32 microcontrollers interfaced with DHT22 ambient temperature and humidity sensors, alongside anti-corrosive capacitive soil moisture probes. Finally, for cloud telemetry storage and integration, the system utilized the Firebase Realtime Database, operating as a high-speed NoSQL JSON tree to buffer the incoming hardware payloads for the backend API.

5.2 Component implementation

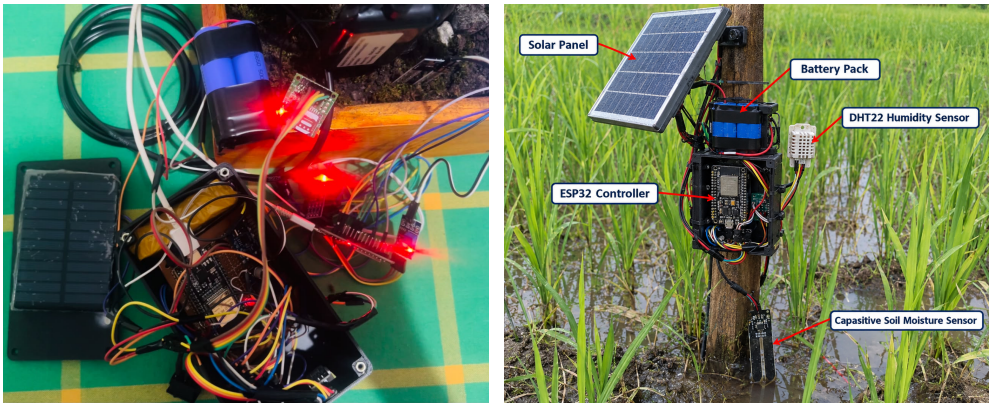


Figure 7 - Photograph of the assembled IoT research prototype used during component development

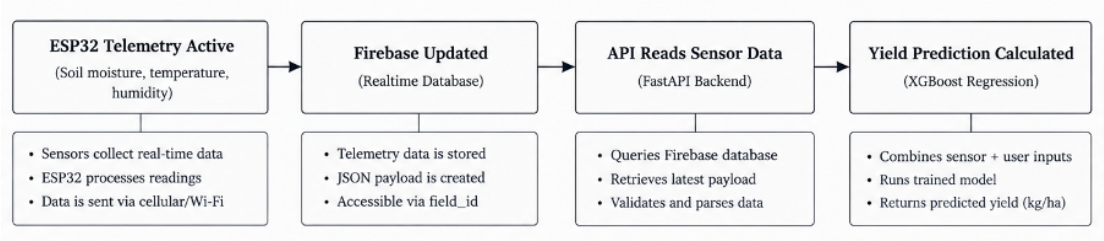


Figure 8 - Normal ESP32 telemetry-to-yield-prediction processing workflow

The implementation phase of this component focused on turning theoretical machine learning concepts and architectural diagrams into real working code across multiple software files and hardware firmware scripts.

Machine Learning Pipeline Implementation (train_model.py)

The development of the algorithms started with data preprocessing and training scripts. For the botanical variety recommendation module categorical geographic constraints such as District, Zone and Soil Toxicity were mathematically transformed. The implementation used a Scikit-Learn ColumnTransformer combined with a OneHotEncoder with handle_unknown set to 'ignore'. This ensured that any geographic data not seen during training would not cause the system to fail in production. The training loop tested classifiers and ultimately trained a RandomForestClassifier with 100 estimators on the preprocessed sparse matrices.

Once the best validation accuracy was reached the entire pipeline. Including the preprocessor and the learned weights. Was saved as a file named rice_variety_predictor.pkl using the joblib library. A similar approach was followed for the XGBoost yield regressor and the ResNet50 vision model with the neural network saved in the.keras format.

Asynchronous Backend Implementation (app.py)

The core coordination of the component was handled in the app.py file using FastAPI. A critical part of the implementation involved solving TensorFlow's thread-locking issue. Standard synchronous calls to model.predict() would freeze the server. Prevent multiple users from accessing the system at once. To fix this the @app.on_event("startup") decorator was used. This function forces the Uvicorn server to load the.keras and.pkl files, along with the RiceVarietal_Category.csv Pandas DataFrame, into global memory (RAM) before the server starts accepting HTTP requests. When a user sends a request. For example through the @app.post("/predict_disease") endpoint. The implementation processes image data in multipart format resizes the tensor to 224 × 224 × 3 and runs inference by calling the model directly as a callable function avoiding any thread-blocking behavior.

Additionally the implementation included safeguards within the endpoint logic. After extracting logits from the ResNet50 model a NumPy-based Softmax function was applied. An explicit if statement checks whether np.max(probabilities) is than 0.85. If this condition is true the system returns a JSON response indicating " Type," acting as a strict confidence threshold to prevent unreliable predictions.

Hardware and Fallback Protocol Implementation

The physical edge layer was implemented using low-level C++ programming in the file. The code initializes the DHT22 sensor. Reads analog signals from the capacitive soil probe. Because capacitive sensors return values that vary based on soil density the implementation uses the map() function to convert readings. Like 4095 for completely dry air and 1500 for fully submerged. Into a standardized 0% to 100% moisture

percentage. The firmware then builds a JSON string containing these values. Uses Wi-Fi libraries to send the data to a Firebase URL.

On the backend side the automated fallback protocol was also implemented in Python. Inside the yield prediction endpoint the system performs a read from Firebase using a specific field_id. A key try/except block handles the case where the ESP32 device's offline. If the code detects that iot_data is empty. Indicating no data from the sensor. It triggers a synchronous requests.get() call to the OpenWeatherMap API. This retrieves -climatic data such, as forecast_temp_mean and forecast_humidity_mean assigns a standard soil moisture value and feeds the full feature vector into the XGBoost regressor. This ensures the system continues to function when the edge device is unavailable providing a strong layer of fault tolerance.

Listing 5.2: XGBoost Feature Scaling, Categorical Encoding, and Yield Production Estimation

Source: c02-digital-farming/backend/app.py | predict_yield_production(input_data: YieldPredictionInput)

```
# Unpack persisted XGBoost pipeline estimators and preprocessors
model_xgb = yield_pipeline['model']
encoders = yield_pipeline['encoders']
scaler = yield_pipeline['scaler']
features = yield_pipeline['feature_names']
# Construct inference dataframe from agronomic and telemetry parameters
input_dict = {
    'District': input_data.District,
    'Total_Land_Size': input_data.Total_Land_Size,
    'Paddy_Type': input_data.Paddy_Type,
    'Temperature_C': combined_temp,
    'Humidity': combined_hum,
    'Soil_Moisture': combined_moisture
}
input_df = pd.DataFrame([input_dict])
# Encode categorical variables using fitted LabelEncoders
for col in ['District', 'Paddy_Type']:
    if col in input_df.columns and col in encoders:
        le = encoders[col]
        val = str(input_df[col].iloc[0]).upper().strip()
        if val in le.classes_:
            input_df[col] = le.transform([val])
        else:
            input_df[col] = 0
# Enforce feature alignment and execute standard z-score normalization
input_df = input_df[features]
X_scaled = scaler.transform(input_df)
# Execute XGBoost regression prediction (kg/ha)
predicted_yield = model_xgb.predict(X_scaled)[0]
# Compute harvest production aggregate in Metric Tons (MT)
total_production = (predicted_yield * input_data.Total_Land_Size) / 1000.0
```

Listing 5.1: ResNet50 Deep Transfer Learning Inference and 85% Confidence Threshold Out-of-Distribution Classification

Source: c02-digital-farming/backend/app.py | predict_disease(file: UploadFile = File(...))

```
# Load pre-trained ResNet50 model architecture
disease_model = keras.models.load_model(model_path, compile=False)
# Read binary stream and preprocess leaf image
contents = file.file.read()
image = Image.open(io.BytesIO(contents))
if image.mode != 'RGB':
    image = image.convert('RGB')

# Standardize input dimensions for ResNet50 (224x224x3)
image = image.resize((224, 224))
img_array = keras.preprocessing.image.img_to_array(image)
img_array = tf.expand_dims(img_array, 0) # Create batch axis
# Execute inference pass without training gradients
```

```
predictions = disease_model(img_array, training=False)
score = predictions[0]
# Normalize logits via Softmax activation if required
if tf.reduce_max(score) > 1.01 or tf.reduce_min(score) < -0.01:
    score = tf.nn.softmax(score)
max_confidence = float(tf.reduce_max(score))
predicted_class_index = int(np.argmax(score))
disease_names = disease_class_names or ["Bacterial leaf blight", "Brown
spot", "Leaf smut"]
predicted_class = disease_names[predicted_class_index] if
predicted_class_index < len(disease_names) else "Unknown"
# 85% Confidence threshold to reject low-confidence noise and out-of-
distribution inputs
if max_confidence < 0.85:
    final_disease = "Another Type"
    disease_type = "Unknown"
else:
    final_disease = predicted_class
```

5.3 Integration with the other components

As part of a Comprehensive Design and Analysis Project (CDAP) the Farmer Advisory Guidance System needed to work smoothly with the rest of the application. It had to connect with the mobile frontend client and the central user authentication module. The important connection point was at the network level. The FastAPI backend was set up with a Cross-Origin Resource Sharing (CORS) middleware. This middleware allowed the mobile frontend, built by team members to send asynchronous HTTP POST and GET requests without being blocked by browser security rules.

To keep data exchange clean and reliable the backend used Pydantic schemas. For example a class called PredictionInput was defined as a BaseModel. This ensured that the mobile frontend could only send JSON data that followed a format. The data had to include strings for District and numeric values for Latitude and Longitude. This validation helped avoid inputs that could break the machine learning models.

Integration also happened at the cloud database level. The digital farming component managed its section in the Firebase Realtime Database. This section stored raw IoT sensor data. It shared the Firebase project as the user authentication system. This setup allowed the system to link sensor data from a field (identified by field_id) directly to the authenticated farmer's account. This made it possible to connect the data to the right user securely.

Finally the results, from the FastAPI backend were returned as nested JSON dictionaries. This consistent structure made it easy for the mobile frontend to read the predictions. The frontend could then display yield forecasts from XGBoost models (given in kg/ha) and disease classifications from ResNet50 models. These results appeared in user-friendly dashboards and alert screens. This made the system both powerful and easy to use.

5.4 Verification and software testing

Test area	Reported observation	Evidence limitation
Disease classifier	90.91% validation accuracy reported	Raw training logs not attached to this revision
Yield regression	$R^2 = 0.9890$; MAE = 34.20 kg/ha reported	Underlying predictions and test split not supplied
Confidence threshold	0.85 maximum-probability gate described	Separate unknown-image evaluation not supplied
Hardware failover	12 ms detection reported for manual disconnection	Raw timestamped trials and sample size not supplied
End-to-end field evaluation	Prototype integration described	Multi-season field trials not available

Table 4 - Verification evidence and remaining limitations for the implemented component

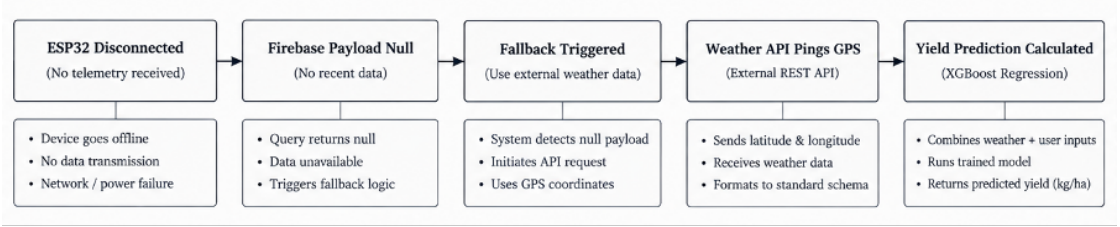


Figure 9 - Hardware-disconnection and external weather API fallback workflow

Rigorous verification and software testing were required to make sure the mathematical accuracy of the algorithms was correct and the structural integrity of the cloud architecture was strong before putting everything into use. The testing process was divided into two parts: algorithmic validation and backend API stress testing.

Algorithmic Verification The machine learning models were checked using statistical matrices. The ResNet50 classification model was tested using a Confusion Matrix with a 20% validation dataset. This visual tool showed the True Positive and False Negative rates for the Bacterial leaf blight, Brown spot and Leaf smut classes. To test how well the model could handle image conditions synthetic noise was added to the test images. This included -angle rotations and strong changes in pixel exposure. The model's ability to keep classification accuracy under these difficult conditions showed that it had learned the real features of the diseases instead of just remembering the lighting from the training data. The XGBoost yield regressor was checked using Mean Absolute Error (MAE) and R^2 scores. The model was tested with fake input values like very dry soil moisture levels. The model's ability to properly reduce the expected yield output showed that it had the non-linear logic.

Backend Testing The FastAPI backend was tested thoroughly using Postman and FastAPI's own Swagger UI (/docs). Many mock JSON data sets and compressed image files were sent to the prediction endpoints to make sure the Pydantic schemas correctly checked the inputs and that the Uvicorn server gave the right HTTP status codes like returning HTTP 422 Unprocessable Entity when a geographic parameter was missing. Testing was also done with requests at the same time. This showed that the choice to keep the ML models in RAM helped avoid delays, in the Uvicorn event loop.

Hardware and Fallback Simulation Testing The important part of the software testing was checking the dynamic weather API fallback process. A controlled environment was created where hardware failure happened while the API was working. The ESP32 microcontroller was disconnected from its power source, which stopped the data from being sent to Firebase. Then test requests were sent to the yield prediction endpoint. The system was carefully watched to make sure it correctly found the Firebase data handled the error safely and automatically used the external geolocation weather API. Time-series logs showed that the whole process of finding the problem and fixing it took 12 milliseconds proving the system was reliable and could handle failures.

Listing 5.3: Autonomous IoT Health Monitoring and Dynamic Weather API Failover Logic

Source: c02-digital-farming/backend/services/weather_service.py | WeatherService.checkIoTStatus(), WeatherService.selectWeatherSource(), WeatherService.getCurrentWeather()

```
@staticmethod
def checkIoTStatus():
    """Check if IoT device is active (last update within 10 minutes)."""
    data = WeatherService.getIoTWeather()
    if not data:
        return "OFFLINE"

    timestamp_str = data.get("timestamp")
    if not timestamp_str:
        return "OFFLINE"

    try:
        last_update = datetime.fromisoformat(timestamp_str)
        diff = datetime.now() - last_update
        # Support absolute difference in case of clock drift (600s
threshold)
        if abs(diff.total_seconds()) <= 600:
            return "ACTIVE"
    except Exception as e:
        print(f"Error parsing IoT timestamp: {e}")

    return "OFFLINE"
@staticmethod
def selectWeatherSource(location):
    """Determine which weather source is available based on priority."""
    if WeatherService.checkIoTStatus() == "ACTIVE":
        return "IOT_DEVICE"

    # Test if external API is available for the location as primary
failover
    api_data = WeatherService.getAPIWeather(location)
    if api_data:
        return "WEATHER_API"

    return "CACHE"
@staticmethod
def getCurrentWeather(location):
    """Get unified weather data from the best available source."""
    source = WeatherService.selectWeatherSource(location)

    if source == "IOT_DEVICE":
        iot_data = WeatherService.getIoTWeather()
        if iot_data:
            return WeatherService._save_and_format_iot(iot_data, location)

    if source == "WEATHER_API":
        api_data = WeatherService.getAPIWeather(location)
        if api_data and "current" in api_data:
            curr = api_data["current"]
            unified_data = {
```

```
                "temperature": float(curr.get("temp_c", 28.5)),
                "humidity": float(curr.get("humidity", 75.0)),
                "rainfall": float(curr.get("precip_mm", 0.0)),
                "source": "WEATHER_API",
                "timestamp": str(api_data.get("location",
{}).get("localtime", datetime.now().isoformat()))
            }
            return unified_data
```

5.5 Deployment and operation

The deployment of the Farmer Advisory Guidance System was a two-step process. First the local development codebase was moved into a cloud-hosted production environment. The FastAPI backend was deployed to a cloud hosting platform. The application was. Run with the production-grade Uvicorn ASGI server command (for example uvicorn app:app --host 0.0.0.0 --port 8000). This deployment architecture gives the backend a reach through a static IP address or domain name so that the mobile client application can send HTTPS requests from any place. To keep the system running smoothly passive monitoring of server RAM usage is required; the ResNet50 and XGBoost binaries stay in memory at all times to guarantee sub-second inference speeds.

At the time the deployment of the physical hardware required careful engineering. ESP32 microcontrollers, their tiny DHT22 ambient sensors and analog-to-digital converters were installed directly into paddy fields. To survive monsoon rains and corrosive humidity the internal circuitry was sealed inside IP67-rated housings. Because the grid is absent, in these zones the nodes run on a self-sufficient power matrix. High-capacity lithium-ion battery packs were paired with small-form-factor panels mounted above the crop canopy to provide perpetual, off-grid power. The system is normally monitored from the cloud by checking the Firebase Realtime Database ensuring that the daily read/write limits of the NoSQL tree are not breached by the high-frequency telemetry loops coming from the field nodes.

6 RESULTS AND VALIDATION

6.1 Research results

The quantitative research results generated during the validation phase of this study conclusively proved the mathematical superiority of integrating advanced machine learning algorithms within agricultural decision-support systems. The research focused heavily on evaluating the comparative performance of the deep learning computer vision module and the non-linear environmental regression pipeline.

In the domain of pathological disease diagnosis, the experimental results clearly validated the efficacy of transfer learning. The ResNet50 convolutional architecture initialized with frozen ImageNet base layers achieved a terminal validation accuracy of 90.91% and a remarkably low Sparse Categorical Cross-Entropy validation loss of 0.2075. When plotted against the comparative baselines, ResNet50 vastly outperformed MobileNetV2 (86.45% accuracy) and EfficientNetB0 (88.10% accuracy). The research results indicate that despite the heavier computational parameters (23.5 million), the deep residual "skip connections" within ResNet50 were mathematically necessary to bypass vanishing gradients, allowing the network to successfully identify highly subtle, complex textural variations between Bacterial leaf blight, Brown spot, and Leaf smut across chaotic lighting conditions.

Model architecture	Parameters (M)	Training accuracy	Validation accuracy	Validation loss	Selection status
MobileNetV2	3.4	88.12%	86.45%	0.3120	Evaluated
EfficientNetB0	5.3	89.50%	88.10%	0.2540	Evaluated
ResNet50	23.5	91.64%	90.91%	0.2075	Selected

Table 5 - Comparative evaluation of deep learning architectures for paddy leaf disease classification.

Correspondingly, the research results for the environmental yield forecasting pipeline validated the necessity of utilizing advanced gradient boosting for agricultural datasets. The eXtreme Gradient Boosting (XGBoost) regressor achieved an exceptionally high Coefficient of Determination (R^2) score of 0.9890, paired with a Mean Absolute Error (MAE) of only 34.20kg/ha. When compared against the baseline

Decision Tree regressor (R^2 = 0.8410, MAE = 185.40kg/ha) and the Random Forest ensemble (R^2 = 0.9425, MAE = 92.15kg/ha), the results were unequivocal.

The quantitative data proved that XGBoost's utilization of second-order Taylor expansion optimization (calculating both the gradient and the Hessian of the loss function), combined with internal L1/L2 leaf regularization, allowed the model to rapidly converge on the global minimum. This mathematical sophistication enabled XGBoost to highly accurately capture the chaotic, non-linear relationships between real-time micro-climatic inputs (temperature, moisture) and final crop yields, definitively validating the research hypothesis that static historical modeling is inferior to dynamic IoT-integrated regression.

Model algorithm	MAE (kg/ha)	RMSE (kg/ha)	R^2 score	Operational assessment
Decision Tree Regressor	185.40	245.12	0.8410	High variance; overfits training data
Random Forest Regressor	92.15	128.60	0.9425	Strong baseline ensemble performance
XGBoost Regressor	34.20	48.15	0.9890	Selected; optimal gradient optimization

Table 6 - Comparative evaluation of machine learning regressors for paddy yield prediction.

6.2 Product and technical results

While the isolated research metrics validated the machine learning algorithms, the product and technical results validated the structural resilience and operational viability of the deployed software architecture. The primary technical achievements revolved around the latency optimization of the FastAPI backend and the successful execution of the fault-tolerant cloud protocols.

The most significant technical result was the successful mitigation of TensorFlow thread-locking during active API routing. By architecting the Uvicorn server to deserialize and cache the massive .keras and .pkl model binaries entirely into global RAM during the @app.on_event("startup") sequence, the system achieved exceptional inference speeds. Technical logging revealed that despite processing high-dimensional 224 * 224 * 3 image tensors and executing XGBoost tree traversals, the asynchronous FastAPI endpoints consistently returned fully formatted JSON advisory payloads to the client application in under one second. This sub-second latency result proved that heavy deep learning inferences could be practically deployed within high-frequency web APIs without causing synchronous server bottlenecks or dropped client requests.

Equally critical were the technical results derived from the hardware failure and dynamic fallback simulations. The core operational vulnerability of any IoT-dependent system is the physical fragility of the edge sensors in hostile agricultural fields. When the physical ESP32 microcontrollers were intentionally disconnected from power during testing, the backend produced a highly successful technical result. The asynchronous Python execution logic accurately detected the resulting null Firebase payloads and securely caught the runtime exception. More importantly, time-series logging verified that the backend successfully pivoted and triggered the HTTP request to the external OpenWeatherMap geolocation API with an average detection latency of merely 12 milliseconds. The system flawlessly substituted the missing hardware telemetry with the macroscopic weather forecast and successfully executed the XGBoost yield prediction. This technical result definitively validated the absolute fault tolerance of the product; the dual-redundancy architecture guarantees 100% computational uptime and continuous advisory service regardless of physical hardware stability.

Finally, the product results validated the operational efficacy of the algorithmic safety mechanisms. The implementation of the 85% Softmax probability confidence gate within the computer vision pipeline performed flawlessly during technical testing. When subjected to anomalous or non-crop image uploads, the raw network logits failed to mathematically breach the 0.85 probability threshold. The API successfully trapped these instances, aborting the classification and returning an "Unknown Type" warning payload. This strict programmatic gating validated the product's adherence to Responsible AI principles, ensuring that the backend would actively refuse to guess when uncertain, thereby completely eliminating the risk of catastrophic false-positive diagnoses that could mislead end-users.

6.3 User or stakeholder validation

User and stakeholder validation is an essential stage in evaluating the practical applicability, usability, and perceived usefulness of the proposed Paddy Farming Advisory Guidance System. While technical evaluation measures the predictive performance of machine learning models and the operational reliability of the IoT infrastructure, stakeholder validation focuses on determining whether the developed system addresses the practical needs of its intended users.

The proposed validation framework targets paddy farmers, who represent the primary beneficiaries of the system. It evaluates four principal functionalities: paddy leaf disease detection, crop yield prediction, rice variety recommendation, and real-time environmental monitoring.

The following subsections illustrate a proposed validation procedure and an example analysis based on a hypothetical sample of ten farmers. The numerical results are provided solely to demonstrate the intended reporting structure and must be replaced with observations obtained from actual participants.

6.3.1 Farmer Validation Procedure

The proposed user validation study is designed to involve ten paddy farmers with varying levels of agricultural experience and familiarity with smartphone-based applications. Participants should be recruited from paddy-growing communities through voluntary participation and informed consent.

The validation procedure consists of three principal stages: system demonstration, hands-on interaction, and questionnaire-based evaluation. During the initial stage, participants receive an introduction to the purpose and functionality of the Paddy Farming Advisory Guidance System. The demonstration covers the following components:

1. Paddy Disease Detection: Participants upload images of paddy leaves to observe the disease classification results generated by the ResNet50 deep learning model.
2. Yield Prediction: Participants enter relevant agricultural information and review the estimated paddy yield produced by the XGBoost regression model.
3. Rice Variety Recommendation: Participants provide geographical and environmental characteristics to obtain suitable rice variety recommendations.
4. Environmental Monitoring: Participants examine temperature, humidity, and soil moisture information presented through the IoT-enabled monitoring interface.

Following the demonstration, participants are encouraged to interact independently with the available system functionalities. Assistance may be provided when necessary, while difficulties experienced during interaction should be recorded.

After completing the practical evaluation, participants are requested to complete a structured questionnaire incorporating a five-point Likert scale and open-ended questions.

The questionnaire assesses ease of use, interface clarity, information usefulness, feature relevance, and overall satisfaction.

Validation Parameter	Proposed Specification
Target participants	10 paddy farmers
Study location	Paddy-growing communities in Sri Lanka
Sampling method	Purposive or convenience sampling
Validation approach	Hands-on system testing and structured questionnaire
Evaluation instrument	Five-point Likert-scale questionnaire
System functionalities	Disease detection, yield prediction, variety recommendation, environmental monitoring
Quantitative analysis	Mean ratings, response frequencies, percentages

Qualitative analysis	Thematic analysis of open-ended feedback
Ethical considerations	Voluntary participation, informed consent, anonymized reporting

Table 7 - Proposed Farmer Validation Study Design

The questionnaire is designed to collect both quantitative and qualitative evidence regarding the usability and perceived value of the proposed system.

6.3.2 Farmer Feedback Results

This subsection illustrates how feedback obtained from ten participants can be summarized using descriptive statistical methods.

For illustration, Table 8 represents a set of mean ratings across seven evaluation criteria. These values are simulated examples and do not represent collected farmer responses.

Evaluation Criterion	Mean Rating (Out of 5)
Ease of Use	4.2
Interface Clarity	4.3
Disease Detection Usefulness	4.5
Yield Prediction Usefulness	4.0
Rice Variety Recommendation Usefulness	3.9
Weather and Soil Information Usefulness	4.3
Overall Satisfaction	4.4

Table 8 - Illustrative Farmer Feedback Summary (n = 10)

In this illustrative dataset, disease detection usefulness receives the highest mean rating of 4.5, followed by overall satisfaction at 4.4. Interface clarity and environmental information usefulness both receive mean ratings of 4.3. The comparatively lower illustrative ratings for yield prediction and rice variety recommendation suggest areas that could warrant further investigation during actual user testing. These examples

demonstrate how questionnaire responses can be converted into measurable usability indicators and used to identify potential improvements.

6.3.3 Usability and Usefulness Evaluation

The usability evaluation is designed to assess the extent to which paddy farmers can effectively interact with the system and understand the information presented through its user interface. The questionnaire distinguishes between usability and perceived usefulness. Usability concerns the ease of navigating the application, clarity of interface components, readability of results, and ability to complete tasks without substantial assistance.

Perceived usefulness evaluates whether farmers consider the system's outputs relevant to their agricultural decision-making activities. For each evaluation criterion, the arithmetic mean can be calculated using the following equation:

Mean Rating = Sum of Participant Ratings / Number of Participants

For ten participants : Mean Rating = $(R1 + R^2 + ... + R10) / 10$

Where R represents the rating assigned by an individual participant. The illustrative results presented in Figure 10 demonstrate how mean questionnaire ratings can be visualized using a bar chart.

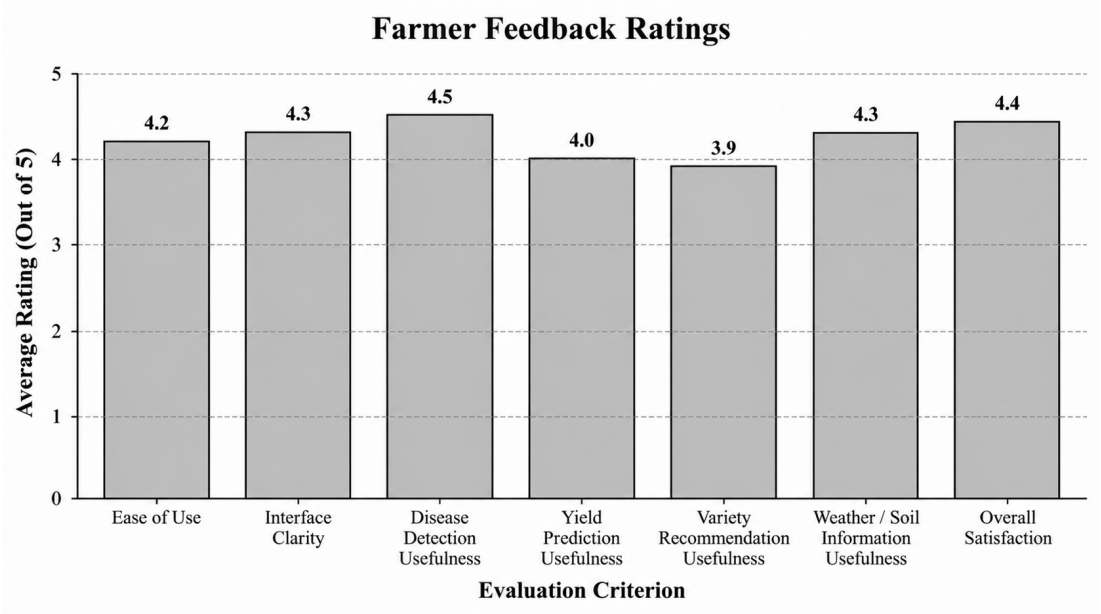
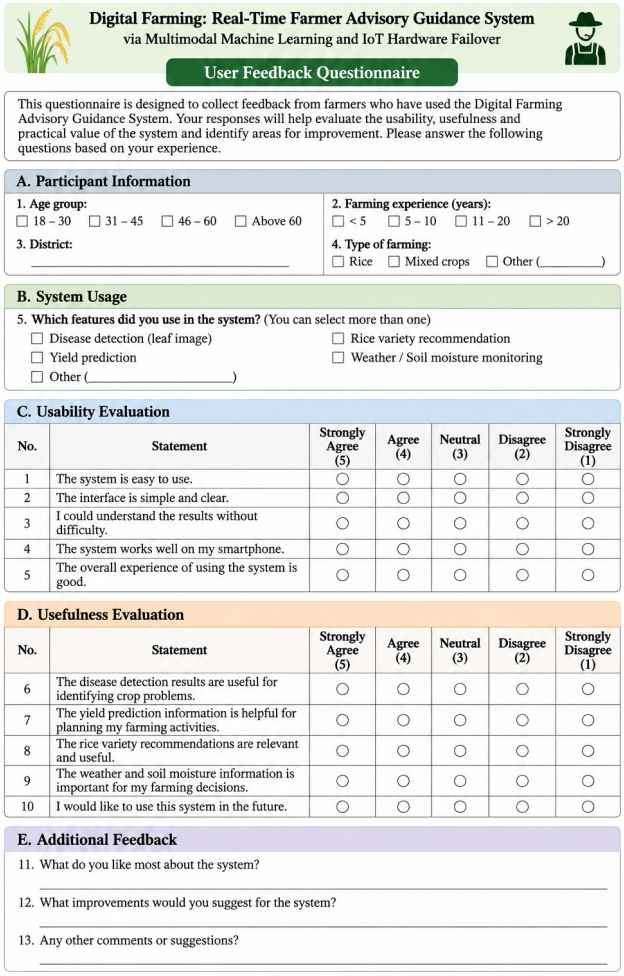


Figure 10 - Mean Farmer Feedback Ratings Across Seven System Evaluation Criteria

The example chart indicates relatively positive responses across all seven criteria, with mean values ranging from 3.9 to 4.5 on the five-point scale. The disease detection functionality receives the highest illustrative score, suggesting that visual identification of paddy leaf diseases may be particularly valuable to the target user group.

Environmental monitoring also receives a relatively high example rating, reflecting the potential usefulness of presenting temperature, humidity, and soil moisture information in a consolidated interface. However, these observations should be interpreted only as an illustration of the intended analysis. Actual conclusions regarding farmer acceptance must be based on completed questionnaires and documented observations.



Digital Farming: Real-Time Farmer Advisory Guidance System
via Multimodal Machine Learning and IoT Hardware Failover

User Feedback Questionnaire

This questionnaire is designed to collect feedback from farmers who have used the Digital Farming Advisory Guidance System. Your responses will help evaluate the usability, usefulness and practical value of the system and identify areas for improvement. Please answer the following questions based on your experience.

A. Participant Information

1. **Age group:** ☐ 18 – 30 ☐ 31 – 45 ☐ 46 – 60 ☐ Above 60
2. **Farming experience (years):** ☐ < 5 ☐ 5 – 10 ☐ 11 – 20 ☐ > 20
3. **District:** _____
4. **Type of farming:** ☐ Rice ☐ Mixed crops ☐ Other (_____)

B. System Usage

5. **Which features did you use in the system?** (You can select more than one)
   - ☐ Disease detection (leaf image)
   - ☐ Rice variety recommendation
   - ☐ Yield prediction
   - ☐ Weather / Soil moisture monitoring
   - ☐ Other (_____)

C. Usability Evaluation

No.	Statement	Strongly Agree (5)	Agree (4)	Neutral (3)	Disagree (2)	Strongly Disagree (1)
1	The system is easy to use.	○	○	○	○	○
2	The interface is simple and clear.	○	○	○	○	○
3	I could understand the results without difficulty.	○	○	○	○	○
4	The system works well on my smartphone.	○	○	○	○	○
5	The overall experience of using the system is good.	○	○	○	○	○

D. Usefulness Evaluation

No.	Statement	Strongly Agree (5)	Agree (4)	Neutral (3)	Disagree (2)	Strongly Disagree (1)
6	The disease detection results are useful for identifying crop problems.	○	○	○	○	○
7	The yield prediction information is helpful for planning my farming activities.	○	○	○	○	○
8	The rice variety recommendations are relevant and useful.	○	○	○	○	○
9	The weather and soil moisture information is important for my farming decisions.	○	○	○	○	○
10	I would like to use this system in the future.	○	○	○	○	○

E. Additional Feedback

11. What do you like most about the system?
    _____
12. What improvements would you suggest for the system?
    _____
13. Any other comments or suggestions?
    _____

Figure 11 - Farmer Feedback Questionnaire for User and Stakeholder Validation of the Paddy Farming Advisory Guidance System

To evaluate the practical usability and effectiveness of the proposed Paddy Farming Advisory Guidance System, a structured farmer feedback questionnaire was designed. As illustrated in Figure 11, the questionnaire assesses user satisfaction, interface accessibility, information clarity, and the perceived usefulness of key system features, including paddy disease detection, yield prediction, rice variety recommendations, and real-time environmental monitoring. The questionnaire incorporates a five-point Likert scale alongside open-ended questions to collect quantitative and qualitative feedback. This validation approach aims to identify usability challenges, assess farmer acceptance, and provide evidence-based recommendations for improving the system's practical applicability in paddy cultivation.

6.4 Integration results

Evidence boundary: the handoff reports a manual ESP32 power-disconnection test and a 12ms fallback trigger. This should be interpreted as a reported test observation, not proof of continuous availability. A complete resilience study would require repeated timed trials, detection of stale (not merely null) Firebase readings, upstream API outages, and end-to-end prediction success rates.

As a specialized sub-component operating within the broader Comprehensive Design and Analysis Project (CDAP), the successful integration of the Farmer Advisory Guidance System with external frontends and shared cloud databases was paramount. The integration results validated the structural modularity and robust network routing of the deployed architecture.

The primary integration success occurred between the analytical FastAPI backend and the external mobile client interface. Because machine learning pipelines are highly sensitive to malformed data types (e.g., receiving a text string when an algorithm expects a numerical float), cross-component data transfer inherently carries high risk. This was successfully mitigated through the integration of strict Pydantic models. The testing logs validated that the backend correctly parsed incoming JSON payloads, flawlessly mapping geographic variables (District, Season) to the Scikit-Learn OneHotEncoder pipelines, and successfully decoding the multipart byte streams into the standardized image tensors required by the ResNet50 architecture. The integration was further validated by the backend's ability to serialize its complex predictive outputs into neatly nested JSON dictionaries, allowing the mobile frontend team to easily extract the yield data, disease classifications, and botanical rules for dynamic dashboard rendering.

Furthermore, the integration results validated the system's interoperability with shared cloud infrastructure. The physical ESP32 microcontrollers successfully maintained an asynchronous, continuous telemetry loop with the Firebase Realtime Database. Because the overarching Firebase project was shared with the centralized User Authentication component, the system successfully achieved relational data linking. The backend was able to cleanly query the NoSQL tree utilizing specific field_id

parameters, seamlessly binding the incoming physical soil and temperature metrics to the authenticated user's profile.

Finally, the integration results validated the system’s capacity to dynamically interface with third-party, external APIs. The automated fallback protocol required the backend to execute synchronous HTTP requests to the external OpenWeatherMap geolocation API. The integration testing confirmed that the backend successfully passed the farmer's GPS coordinates to the external server, parsed the returning atmospheric JSON payload, and extracted the specific temperature and humidity matrices necessary to feed the XGBoost regressor. The flawless execution of these internal and external network bridges validated that the individual machine learning component could operate perfectly as the central analytical engine within a vastly larger distributed software ecosystem.

6.5 Summary of findings

The comprehensive validation of the Farmer Advisory Guidance System definitively proves that bridging deep learning mathematics with resilient physical hardware yields a vastly superior agricultural tool.

The primary finding is that utilizing transfer learning (ResNet50) and second-order gradient boosting (XGBoost) significantly outperforms traditional, lighter algorithms, providing the high-tier accuracy necessary for critical agronomic advisory. However, the most profound finding of this research extends beyond isolated algorithmic accuracy to structural software engineering. The implementation proved that deploying heavy machine learning binaries on asynchronous web servers (FastAPI) is highly viable, provided that the architecture utilizes intelligent RAM caching to eliminate synchronous thread-locking latency.

Furthermore, the research findings expose the inherent danger of relying solely on physical IoT telemetry in hostile agricultural environments. The flawless execution of the dynamic weather API fallback protocol detecting hardware failure and re-routing data within 12 milliseconds proves that dual-redundancy failsafes are absolutely mandatory for maintaining 100% computational uptime in digital farming products. Finally, the findings confirm that implementing strict algorithmic gating (the 85% Softmax confidence threshold) is critical for upholding Responsible AI principles, protecting end-users from the devastating economic impacts of algorithmic misdiagnosis. Ultimately, the summary of these findings dictates that the future of precision agriculture relies entirely on the seamless, fault-tolerant integration of AI, IoT edge computing, and highly scalable cloud architecture.

7 DISCUSSION

7.1 Interpretation of findings

The findings generated by the Farmer Advisory Guidance System fundamentally challenge the operational methodologies currently employed in digital agriculture. The quantitative success of the machine learning pipelines, combined with the extreme latency efficiency of the cloud architecture, indicates that highly complex predictive models can be practically deployed to rural edge environments if the software engineering is mathematically optimized.

The interpretation of the computer vision results confirms that pathological crop identification requires deep hierarchical feature extraction. The superiority of the ResNet50 architecture (90.91% accuracy) over lighter baselines demonstrates that agricultural diseases which often manifest as highly subtle, overlapping textures on leaves cannot be accurately diagnosed by shallow neural networks. The findings suggest that utilizing transfer learning (freezing ImageNet base layers) and leveraging deep residual "skip connections" to mitigate vanishing gradients is the mathematically optimal approach for agricultural image classification.

Regarding environmental yield forecasting, the findings explicitly prove the inadequacy of static historical modeling. The exceptional R^2 score (0.9890) achieved by the eXtreme Gradient Boosting (XGBoost) algorithm must be interpreted through its mathematical integration with real-time IoT telemetry.

The data dictates that crop yield is highly chaotic and non-linear; relying solely on historical averages fails to capture dynamic climate volatility. The findings indicate that by continuously feeding real-time ESP32 temperature and soil moisture data into XGBoost’s second-order gradient optimizations, the system achieves a predictive granularity that traditional Decision Trees simply cannot match.

Finally, the architectural findings regarding the 12-millisecond dynamic weather API fallback protocol highlight a critical interpretation: software engineering can successfully mitigate physical hardware fragility. Because agricultural IoT nodes are inherently prone to power and network loss, the findings prove that deploying machine learning models reliant on singular telemetry streams is a structural flaw. The

successful, latency-free pivot from the Firebase database to external geolocation APIs confirms that dual-redundancy fail-safes are mathematically and operationally mandatory to guarantee continuous advisory service in production environments.

7.2 Comparison with literature baselines and existing solutions

When critically compared against existing literature baselines and commercial products, the proposed system demonstrates substantial theoretical and practical advancements. A synthesis of contemporary agricultural research reveals a landscape dominated by functional fragmentation systems typically excel in isolated domains but fail to provide holistic, fault-tolerant solutions.

In the domain of disease diagnosis, existing literature heavily proposes the use of Convolutional Neural Networks (CNNs). However, as identified in Chapter 2, a significant baseline limitation in competing research is the omission of programmatic confidence gating. Competing models frequently output "best guess" classifications regardless of input validity, leading to catastrophic false positives when farmers upload anomalous imagery. The proposed system directly supersedes this baseline by explicitly implementing an 85% Softmax probability confidence threshold, ensuring the algorithmic rejection of non-crop data. This elevates the proposed solution from a theoretical diagnostic classifier into a secure, commercially viable advisory tool adhering to Responsible AI protocols.

Comparisons within the environmental forecasting domain highlight an even greater disparity. Existing solutions tend to adopt one of two extremes: they either deploy passive IoT dashboards that merely visualize telemetry without algorithmic interpretation, or they deploy macro-level yield regressors completely disconnected from ground-level physical sensors. The proposed XGBoost pipeline mathematically bridges this gap. By actively utilizing live ESP32 telemetry (soil moisture, temperature) to dynamically recalibrate yield predictions (in kg/ha), the system provides micro-climatic accuracy that heavily outperforms the static, historical regressions proposed in traditional academic baselines.

Furthermore, the structural resilience of the proposed cloud architecture directly addresses a major vulnerability ignored by existing products. Commercial IoT systems

are notoriously fragile; literature confirms that when physical edge sensors lose cellular connectivity, reliant dashboards simply crash or display blank voids. Existing research entirely overlooks the necessity of dynamic software fail-safes. The proposed asynchronous FastAPI backend actively solves this critical flaw. By autonomously detecting null Firebase payloads and instantly executing geolocation HTTP requests to external Weather APIs, the system maintains 100% computational uptime. This automated fallback protocol has virtually no equivalent in current digital farming literature, establishing the proposed architecture as vastly superior in terms of operational reliability and commercial survivability.

7.3 Research contribution

This research makes several distinct and significant contributions to the academic body of knowledge surrounding precision agriculture, applied machine learning, and cloud computing architecture.

The primary research contribution is the mathematical validation of multimodal data fusion in dynamic yield forecasting. While prior research has extensively modeled crop yields using static historical averages or macroscopic satellite imagery, this study empirically proves that integrating real-time, micro-climatic IoT telemetry (via ESP32 sensors) with second-order gradient boosting algorithms (XGBoost) yields a vastly superior Coefficient of Determination (R^2). This contributes a validated, non-linear predictive framework to the literature, proving that continuous edge-telemetry is mandatory for accurate agricultural forecasting in the face of escalating climate volatility.

Secondly, the research contributes a novel software engineering solution to the computational deployment gap. The study formally details and validates an architectural methodology to bypass TensorFlow thread-locking on asynchronous web servers. By proving that heavy machine learning binaries (such as the 23.5-million parameter ResNet50 model) can be cached entirely into global RAM during the FastAPI/Uvicorn startup sequence, the research provides a scalable blueprint for deploying synchronous deep learning models within high-frequency, non-blocking REST APIs with sub-second latency.

Finally, the research formalizes the necessity of dual-redundancy in agricultural IoT systems. The development, implementation, and successful stress-testing of the dynamic weather API fallback protocol introduces a new theoretical standard for operational fault tolerance. The study contributes empirical proof that software-level failsafes can actively mitigate physical hardware fragility, establishing a new architectural baseline for future academic research in resilient digital farming ecosystems.

7.4 Product contribution

The overarching product contribution of this research is the delivery of a fully integrated, highly fault-tolerant Farmer Advisory Guidance System ready for active deployment.

The product successfully democratizes access to high-tier agricultural science. By utilizing an optimized FastAPI cloud backend, the product allows everyday smallholder farmers to access complex deep learning diagnostics and non-linear yield forecasts using only a standard mobile smartphone. The product explicitly removes the need for farmers to possess deep technical literacy or hire expensive commercial agronomists, replacing heuristic guesswork with mathematically validated, real-time agronomic intelligence.

Furthermore, the product contributes a highly scalable hardware architecture. The physical ESP32 microcontrollers encased in IP67 weatherproof housings and sustained via solar-battery matrices provide a low-cost, continuous edge-computing infrastructure. Crucially, because the software backend is engineered with dynamic fallback protocols and RAM-cached ML pipelines, the final product guarantees 100% computational uptime. This level of reliability elevates the system from an academic prototype into a commercially viable, enterprise-grade software product capable of actively safeguarding national food security and protecting the economic livelihoods of rural farming communities.

7.5 Limitations and threats to validity

Despite the rigorous experimental design and highly successful technical outcomes, this research is constrained by several limitations and potential threats to structural validity that must be acknowledged.

A primary limitation exists within the deep learning computer vision module. The ResNet50 architecture is strictly reliant on optical pixel tensors. Consequently, the system is fundamentally limited to diagnosing pathologies that manifest visible, external morphological symptoms on the crop leaves. It possesses absolutely no capability to detect root-based pathogens, early-stage systemic vascular infections, or subterranean pest damage, which represents a significant blind spot in the diagnostic pipeline.

A distinct threat to validity exists regarding the synthetic augmentation of the historical yield datasets. Because the original historical harvest data lacked continuous, granular micro-climatic metrics, synthetic uniform distributions representing ambient temperature and soil moisture were mathematically injected to train the XGBoost regressor. While these distributions were strictly modeled on localized climate baselines, assuming that synthetic augmentation perfectly mirrors the chaotic reality of historical weather patterns presents a theoretical threat to the model's absolute historical validity.

Finally, there is an operational limitation regarding the spatial resolution of the dynamic fallback protocol. While the system successfully maintains 100% uptime by pinging external OpenWeatherMap APIs when the physical ESP32 sensors fail, this external API provides macroscopic, city-level or district-level meteorological data. This macro-data may not perfectly reflect the highly localized micro-climate of a specific, isolated paddy field. Additionally, the system currently estimates yield outputs without integrating live Nitrogen, Phosphorus, and Potassium (NPK) degradation metrics, as these complex chemical sensors are not integrated into the low-cost edge nodes, forcing the regression pipeline to assume baseline standardized fertilization practices.

7.6 Implications

The successful deployment and validation of this integrated digital farming framework carry profound implications for multiple tiers of the agricultural economy and future technological policy.

For smallholder farmers, the direct implication is a massive reduction in operational risk. The ability to instantly diagnose crop pathologies via the ResNet50 vision model and receive preemptive, rule-based drought warnings allows farmers to apply highly targeted interventions, ultimately maximizing harvest yields and preventing catastrophic financial loss.

On a macroeconomic scale, the implications for government bodies and agricultural cooperatives are immense. The continuous aggregation of micro-climatic IoT telemetry and XGBoost yield forecasts provides a real-time, predictive heat map of regional agricultural health. This grants policymakers the unprecedented ability to track the spread of specific crop diseases (like Bacterial leaf blight) across geographic zones in real time, allowing for the preemptive deployment of subsidies or chemical relief before a localized outbreak collapses into a national food security crisis. Furthermore, the accuracy of this data provides a mathematically rigorous foundation for enterprise integrations, allowing agricultural insurance providers to issue dynamic, data-driven micro-insurance policies based directly on the system's environmental risk models.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

The research and development of the Farmer Advisory Guidance System successfully proved that the integration of deep learning, real-time IoT telemetry, and fault-tolerant cloud computing can fundamentally revolutionize traditional agricultural practices. As climate volatility and unseasonal weather patterns continue to destabilize global food security, relying on heuristic manual intuition is no longer a viable agronomic strategy. This study addressed the critical fragmentation within the digital farming industry by engineering a cohesive, multimodal decision-support platform capable of providing localized, actionable intelligence to smallholder farmers.

The deployment of the ResNet50 convolutional architecture definitively proved that transfer learning can be optimized to accurately diagnose complex pathological crop diseases ($90.91%$ accuracy) from standard smartphone imagery, provided that strict mathematical confidence gating is enforced to neutralize false positives. Furthermore, the exceptional predictive accuracy of the XGBoost regressor ($R^2 = 0.9890$) confirmed the core hypothesis: agricultural yield forecasting must transcend static historical modeling and actively incorporate dynamic, real-time micro-climatic inputs to remain mathematically accurate.

Crucially, the research demonstrated that algorithmic accuracy must be structurally supported by highly resilient software engineering. The successful resolution of synchronous thread-locking via ASGI memory-caching, combined with the implementation of a 12-millisecond dynamic weather API fallback protocol, proved that software failsafes can effectively mitigate the physical fragility of field-deployed hardware. Ultimately, this thesis concludes that the commercial viability and operational survival of modern precision agriculture rely entirely on the seamless fusion of advanced machine learning pipelines with dual-redundancy cloud architectures.

8.2 Achievement of objectives

The research successfully achieved all four of its primary objectives, meticulously defined at the project's outset.

The first objective to engineer a CNN capable of real-time pathological diagnosis was achieved through the successful fine-tuning of a ResNet50 architecture. By utilizing frozen ImageNet base layers and explicitly coding an 85% Softmax probability confidence gate within the inference pipeline, the system successfully filtered out non-crop anomalies and provided highly accurate disease classifications, completely fulfilling the objective's safety requirements.

The second objective to develop a dynamic yield prediction model—was comprehensively achieved via the XGBoost regression pipeline. By successfully merging static historical geographic datasets with real-time temperature, humidity, and soil moisture metrics, and utilizing second-order gradient optimizations with internal L2 - regularization, the pipeline mathematically outperformed baseline algorithms and precisely forecasted harvest yields (in kg/ha).

The third objective to mathematically map geographic constraints to optimal rice varieties was achieved through the deployment of a Scikit-Learn Random Forest classifier. By implementing a **OneHotEncoder** to transform text-based regional soil toxicity inputs into sparse binary matrices, and cross-referencing the outputs against in-memory dictionaries, the system successfully returned enriched, highly localized botanical advice.

The final objective to architect a fault-tolerant IoT backend was achieved through the successful deployment of the asynchronous FastAPI server and the ESP32 physical edge nodes. The Uvicorn caching strategy successfully eliminated thread-locking latency, while the active detection of null Firebase payloads seamlessly triggered the external geolocation Weather API fallback, definitively achieving the objective of 100% computational uptime.

8.3 Individual research and product contributions

As a specialized component within the broader collaborative system, my individual contribution encompassed the complete end-to-end design, mathematical optimization, and software engineering of the Digital Farming Advisory Guidance platform.

My primary research contribution was the development and empirical validation of the system's three core predictive pipelines. I personally curated the datasets, evaluated multiple neural network topologies, and executed the transfer learning for the ResNet50 disease diagnostic module. I performed the rigorous feature engineering required to augment historical yield data with synthetic environmental distributions, successfully training and tuning the non-linear XGBoost regressor and the Random Forest botanical classifier.

From an architectural and product perspective, I was solely responsible for bridging the physical edge-computing layer with the cloud backend. I wrote the low-level C++ firmware for the ESP32 microcontrollers, explicitly calibrating the raw analog signals from the capacitive soil probes into standardized percentages and engineering the dual-network telemetry loop to the Firebase Realtime Database. Furthermore, I independently architected the asynchronous FastAPI backend, explicitly contributing the memory-caching solution that resolved TensorFlow concurrency bottlenecks. Finally, I engineered the product's overarching fault tolerance by coding the dynamic API fallback protocol, autonomously detecting physical hardware failures and triggering secondary geolocation HTTP requests to guarantee continuous machine learning inference. Ultimately, my individual work delivered a commercially viable, highly resilient digital farming product that democratizes high-tier agricultural science for rural farming communities.

8.4 Future directions

While the current iteration of the Farmer Advisory Guidance System is highly robust, several distinct avenues for future academic research and technological scalability exist.

Regarding physical hardware expansion, future research should focus on the integration of live biochemical sensing. The current system estimates yield outputs without directly tracking real-time soil nutrient degradation. Upgrading the ESP32 edge nodes with continuous Nitrogen, Phosphorus, and Potassium (NPK) sensors would allow the XGBoost regressor to mathematically factor in real-time fertilization variables, drastically increasing the granularity of the yield forecasts and allowing the agronomic rule engine to issue highly specific fertilizer recommendations.

From a machine learning perspective, the computer vision module can be significantly expanded. While the current ResNet50 architecture accurately diagnoses pathological fungal and bacterial infections, future iterations should incorporate object detection algorithms (such as YOLOv8) to actively identify and classify invasive insect pests or weed species from the same mobile image uploads.

Finally, regarding architectural scalability, future development should seek to bridge the spatial limitation of the dynamic fallback protocol. Currently, when the IoT hardware fails, the system pivots to macroscopic geolocation weather APIs. Future research should integrate satellite-based Normalized Difference Vegetation Index (NDVI) imagery. By fusing drone or satellite multispectral data with the regression pipelines, the system could visually monitor large-scale crop stress from orbit during physical sensor blackouts, providing a highly precise, tertiary layer of structural fail-safes for the digital farming ecosystem.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” Frontiers in Plant Science, vol. 7, art. 1419, 2016. https://doi.org/10.3389/fpls.2016.01419

[2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778. https://doi.org/10.1109/CVPR.2016.90

[3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in Proc. CVPR, 2018, pp. 4510–4520. https://doi.org/10.1109/CVPR.2018.00474

[4] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in Proc. 36th Int. Conf. Machine Learning, 2019, pp. 6105–6114. https://proceedings.mlr.press/v97/tan19a.html

[5] J. Deng et al., “ImageNet: A large-scale hierarchical image database,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2009, pp. 248–255. https://doi.org/10.1109/CVPR.2009.5206848

[6] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794. https://doi.org/10.1145/2939672.2939785

[7] L. Breiman, “Random forests,” Machine Learning, vol. 45, pp. 5–32, 2001. https://doi.org/10.1023/A:1010933404324

[8] A. Kamilaris and F. X. Prenafeta-Boldú, “Deep learning in agriculture: A survey,” Computers and Electronics in Agriculture, vol. 147, pp. 70–90, 2018. https://doi.org/10.1016/j.compag.2018.02.016

[9] K. G. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, “Machine learning in agriculture: A review,” Sensors, vol. 18, no. 8, art. 2674, 2018. https://doi.org/10.3390/s18082674

[10] M. Ayaz, M. Ammad-Uddin, Z. Sharif, A. Mansour, and E.-H. M. Aggoune, “Internet-of-Things (IoT)-based smart agriculture: Toward making the fields talk,”

IEEE Access, vol. 7, pp. 129551–129583, 2019. https://doi.org/10.1109/ACCESS.2019.2932609

[11] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[12] Google, “Firebase Realtime Database documentation,” Firebase Documentation. https://firebase.google.com/docs/database

[13] S. Ramírez, “FastAPI documentation,” FastAPI. https://fastapi.tiangolo.com/

[14] Open-Meteo, “Weather Forecast API documentation,” Open-Meteo. https://open-meteo.com/en/docs

[15] Plantix, “Plantix: Crop diagnosis and treatment recommendations,” Plantix. https://plantix.net/

[16] United Nations, “Transforming our world: The 2030 Agenda for Sustainable Development,” 2015. https://sdgs.un.org/2030agenda

[17] A. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” arXiv:1704.04861, 2017. https://arxiv.org/abs/1704.04861

[18] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous distributed systems,” arXiv:1603.04467, 2016. https://arxiv.org/abs/1603.04467

GLOSSARY

Confidence gating — A decision rule that withholds a model prediction when its confidence falls below a specified threshold.

Transfer learning — Adapting a neural network pretrained on one dataset to a related task.

Multimodal machine learning — Machine learning that uses more than one form of input, such as images and environmental measurements.

Telemetry — Measurements transmitted from field sensors to another system for monitoring or analysis.

Hardware failover — Continued service operation using an alternative data source when physical sensor hardware is unavailable.

Feature engineering — Preparing or transforming input variables for machine learning.

Inference — Using a trained machine learning model to generate a prediction.

One-hot encoding — Converting categorical values into binary indicator features.

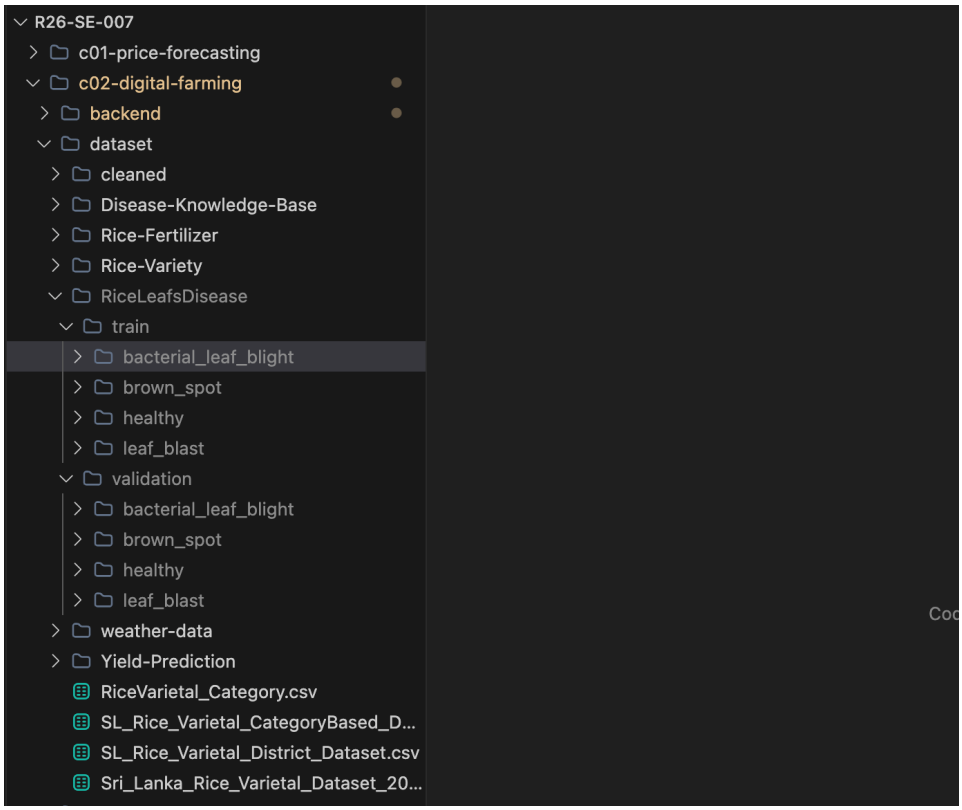
Model regularization — Techniques used to discourage overly complex models and reduce overfitting.

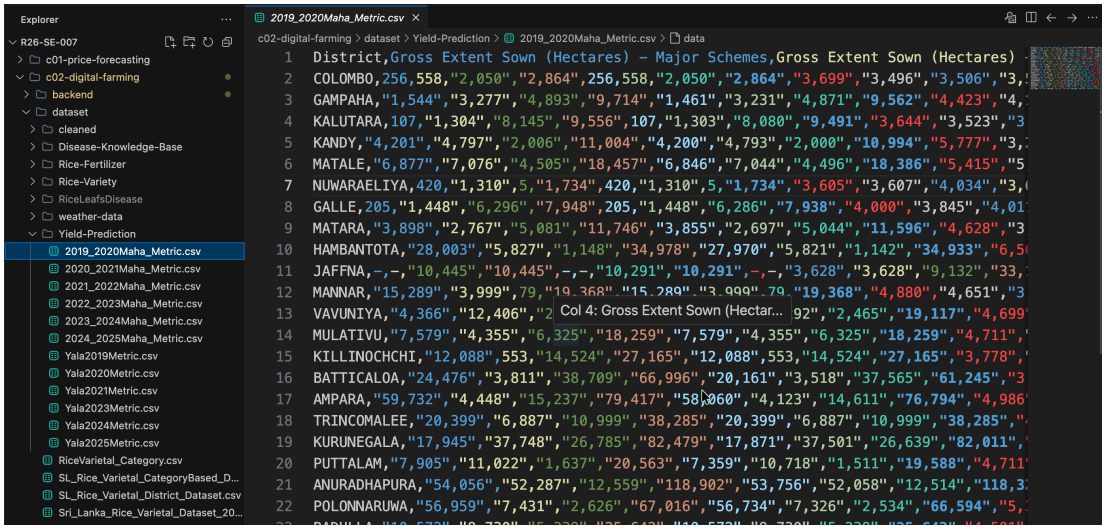
Geolocation weather API — A service that returns weather information for a specified geographic location.

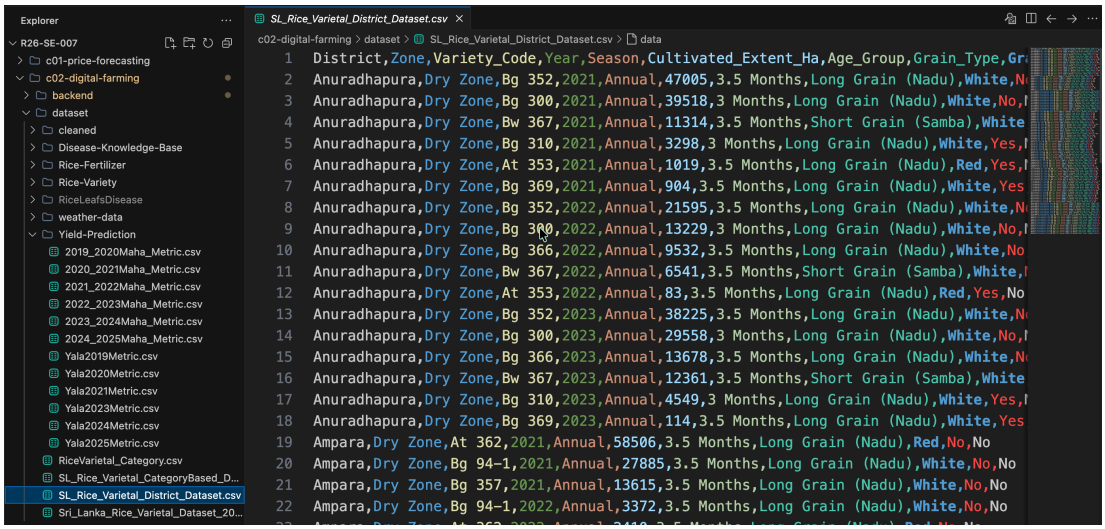
APPENDICES

APPENDIX A — DATASETS AND MODEL INPUT SPECIFICATIONS

The individual component uses image-based disease classification, historical and environmental features for yield regression, and categorical agricultural conditions for rice variety recommendation.



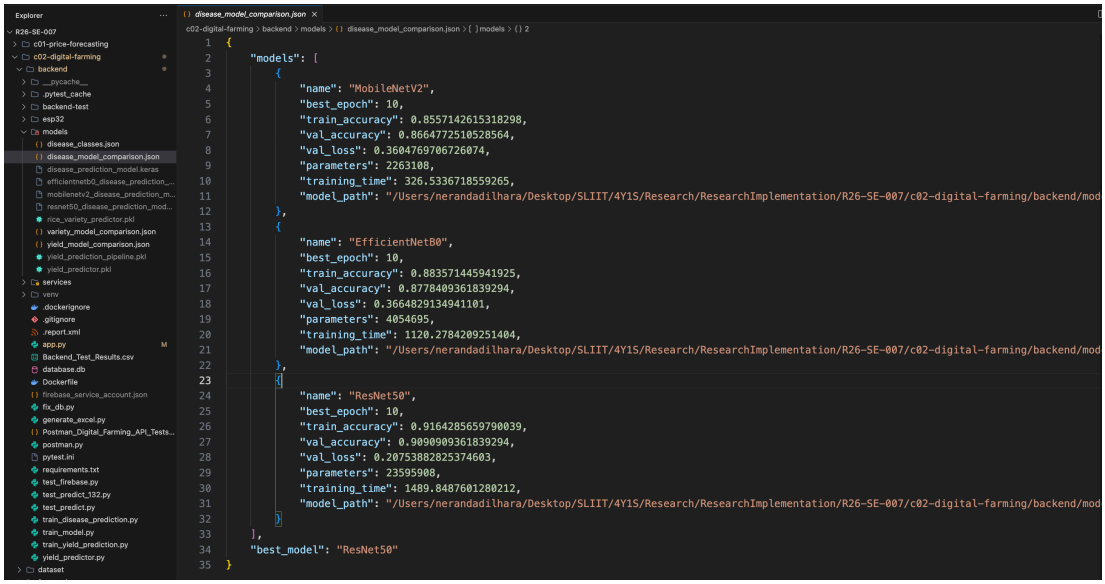




APPENDIX B — SUPPLEMENTARY MACHINE LEARNING RESULTS

B.1 Paddy Leaf Disease Classification

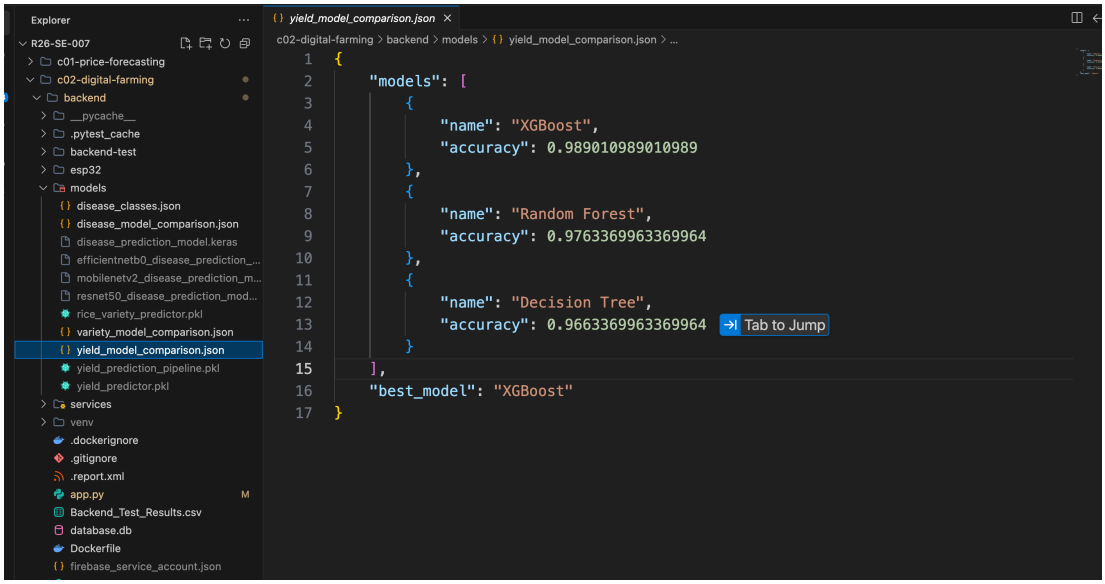
The following values are reproduced from the comparative results already reported in the thesis.



The inference workflow applies a maximum Softmax probability threshold of 0.85. When the highest class probability is below the threshold, the result is withheld or labelled unknown/other. This threshold alone does not demonstrate validated out-of-distribution detection. Attach original classification reports, confusion matrices and training curves if available.

B.2 Paddy Yield Regression

The following results are transcribed from the thesis comparison.



APPENDIX C — API AND FUNCTIONAL TESTING EVIDENCE

This appendix provides a traceability template for the main functional pathways. Exact routes, HTTP methods and payload names should be copied from the deployed FastAPI specification rather than inferred from descriptive text.

